

Artificial Intelligence Project #3 Reinforcement Learning Report

414551003 Yong-Cheng Liaw¹

¹National Yang Ming Chiao Tung University

1 Research Question and Motivation

Deep reinforcement learning can appear successful when an agent reaches a high score once, but a single peak performance does not necessarily mean that the agent has learned a policy that remains effective over time. My main motivation is that a reinforcement learning method should not only discover a good policy, but also retain it. This report studies that gap through the DQN family. The central research question is: **which components of a DQN agent are most important for learning and retaining effective behavior across tasks with very different observation and reward structures?**

To answer this question, I compare six DQN variants on Lunar Lander and Atari Freeway. Lunar Lander is a compact control task with shaped rewards, while Freeway is a sparse-reward visual Atari task where progress depends on repeated successful crossings. This contrast makes the experiments more informative than a single-task comparison, because the same algorithm may fail through unstable control in one environment and through forgetting sparse-reward behavior in the other.

Rather than focusing only on final score, I examine whether an agent reaches a useful performance threshold, whether it sustains that threshold, how much performance is lost between the best checkpoint and the final checkpoint, and what the final learned behavior looks like. In each section, I use tables, learning curves, behavior figures, and cross-task summaries to show how replay buffers, target networks, Double DQN, and Dueling DQN affect learning stability and long-term policy retention.

2 Research Method

2.1 AI Assistance

I used Codex with GPT-5.5 as programming assistance during development. The AI tool helped produce skeleton code and suggest implementation structure. I then refined each component manually, checked the model design and training logic against the project requirements, and adjusted the experiments for the final analysis.

For the report part, I first digest the results and come up with insights, then use AI to transform my insights into figures using matplotlib. I then write descriptions of the insights and use AI to improve my grammar and wording.

2.2 Task Selection

I chose two tasks that expose different parts of the same DQN design. LunarLander-v3 is a compact control task from the Gymnasium Box2D suite. It uses an eight-value state vector and four discrete engine actions, so the agent must learn smooth control from a small input space. Atari Freeway is an image-based Atari task built on the Arcade Learning Environment. It uses screen observations and sparse score rewards, so the agent must learn from visual input and preserve a useful crossing policy over time [1, 2, 3, 4].

This pairing gives the report a simple contrast. Lunar Lander has shaped rewards that respond to distance, speed, angle, contact, and fuel use, so failures often show up as unstable landings or inefficient engine use. Freeway gives a point only when the controlled character reaches the top of the road, so failures often show up as no progress, unsafe crossings, or a policy that stops moving upward. The same algorithm family is therefore tested in both a dense, low-dimensional setting and a sparse, visual setting.

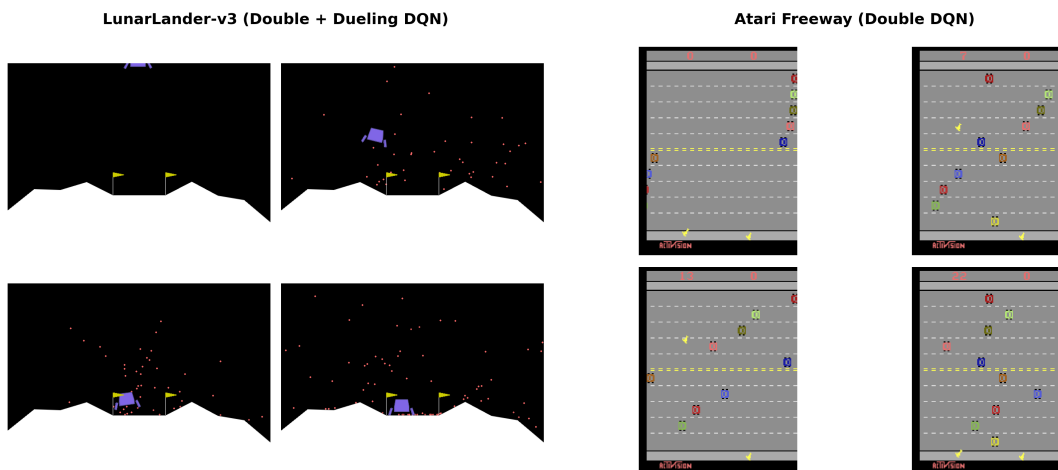


Figure 1: Task snapshots from selected trained agents. The four snapshots on the left show the Lunar Lander agent approaching the landing zone and successfully settling on the platform between the flags. The four snapshots on the right show the Atari Freeway agent crossing traffic without being hit. In Freeway, the trained agent score is shown on the top left and the opponent score on the top right; the left score increases as the trained agent completes crossings.

Table 1: Selected tasks for the DQN ablation study.

Task	Observation	Action Space	Success Signal	Reason for Selection
Lunar Lander	Vector state with position, velocity, angle, and leg contacts.	no-op, left engine, main engine, right engine.	Episode return at least 200.	Shaped rewards and a compact state make it useful for studying stability and sample efficiency.
Atari Freeway	Stacked 84 by 84 grayscale Atari frames after preprocessing.	no-op, up, down.	Score at least 15; one point is earned for each crossing.	Sparse visual rewards make it useful for testing representation learning and policy retention.

2.3 RL Algorithm Selection and Ablation Design

Both selected tasks have discrete action spaces, so I used the DQN family as the common algorithmic base. DQN learns an action-value function $Q(s, a)$ with a neural network and selects actions from the learned value estimates. This is a natural fit for Lunar Lander because the state is a compact vector and the action set is small. It is also a natural fit for Freeway because the original Atari DQN setting was designed for image observations and discrete joystick actions [5].

The baseline DQN design has two important stabilizers. The replay buffer stores past transitions and samples minibatches from mixed time points, which reduces the correlation between consecutive updates and allows each useful transition to be reused. The target network is a delayed copy of the online network. It makes the Bellman target change more slowly, so the network is not always chasing a target produced by its newest parameters. The two ablations remove these mechanisms one at a time. I expected the no-replay variant to be the most fragile because it learns from recent correlated experience. I expected the no-target variant to still learn sometimes, especially in the smaller Lunar Lander state space, but to show more oscillation and weaker retention.

The remaining variants test two common DQN extensions. Double DQN uses the online network to choose the next action and the target network to evaluate that action, which is intended to reduce overestimation from the max operator [6]. Dueling DQN separates the value of a state from the relative advantage of each action, which should help when many actions have similar value in a state [7]. The final variant combines Double DQN and Dueling DQN to test whether the two ideas are complementary rather than only useful in isolation.

2.4 Implementation

The project implementation uses PyTorch for the Q-networks and optimization, Gymnasium for environment interaction, and ALE through Gymnasium for Atari Freeway. The training code keeps a separate online network and, when the variant uses it, a target network. Each environment step is converted into a transition record. The replay-buffer variants store these transitions and sample minibatches with NumPy random generators, while the no-replay variants update from the newest transition only. The Double DQN variants use the online network to choose the next action and the target network to evaluate it, following the idea described in the previous subsection. The Dueling variants keep the same feature extractor but replace the single Q-value head with separate value and advantage heads. The detailed version information for the main libraries is Python 3.13.3; PyTorch 2.12.0+cu130; Gymnasium 1.3.0; NumPy 2.4.6; ALE 0.11.2;

The Lunar Lander environment is used directly as a vector-input control task. The Freeway environment uses Atari preprocessing before training: frames are converted to grayscale, resized to 84×84 , repeated with frame skip, clipped by reward sign, and stacked into four-frame observations. This makes the input format close to the standard Atari DQN setting while keeping the action space small.

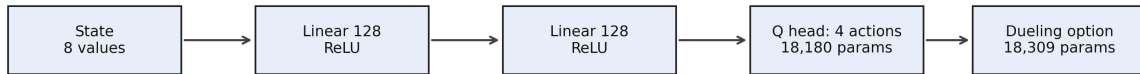
2.5 Model, Training, and Experiment Settings

Algorithm component matrix

DQN	Yes	Yes	No	No
DQN without target network	No	Yes	No	No
DQN without replay buffer	Yes	No	No	No
Double DQN	Yes	Yes	Yes	No
Dueling DQN	Yes	Yes	No	Yes
Double + Dueling DQN	Yes	Yes	Yes	Yes
	Target network	Replay buffer	Double update	Dueling head

Figure 2: Component matrix for the six DQN variants.

Lunar Lander vector Q-network



Atari Freeway visual Q-network



Both models use ReLU activations, orthogonal initialization, and optional mean-subtraction dueling heads.

Figure 3: Network architecture overview for the vector Lunar Lander model and the visual Freeway model.

The same six algorithm variants from Figure 2 were used for both tasks. Lunar Lander was run with seeds 0 through 9 for 500,000 environment steps per run. Freeway was run with seeds 0 through 3 for 1,000,000 agent steps per run, which corresponds to about 4,000,000 ALE frames because the Atari preprocessing uses frame skip 4. Evaluation used 20 episodes every 10,000 steps, and checkpoints were saved every 50,000 steps.

Figure 3 shows the two network families. Lunar Lander uses a small MLP because the observation is already an eight-value state vector. Freeway uses a convolutional network because the observation is a stack of image frames. For the Dueling variants, the final prediction layer is replaced by value and advantage streams with mean-subtraction aggregation. The rest of the training settings are listed in Table 2.

Table 2: Model, training, and experiment settings for the selected result folders.

Setting	Lunar Lander	Atari Freeway
Variants and seeds	DQN, no-target, no-replay, Double DQN, Dueling DQN, and Double + Dueling DQN. Seeds 0-9.	DQN, no-target, no-replay, Double DQN, Dueling DQN, and Double + Dueling DQN. Seeds 0-3.
Training budget	500,000 environment steps.	1,000,000 agent steps; 4,000,000 estimated ALE frames.
Evaluation and checkpoints	20 evaluation episodes every 10,000 steps; checkpoints every 50,000 steps; evaluation epsilon 0.001.	20 evaluation episodes every 10,000 steps; checkpoints every 50,000 steps; evaluation epsilon 0.001.
Input preprocessing	Raw 8-value state vector; no reward clipping.	84 by 84 grayscale frames; frame skip 4; no-op max 30; four-frame stack; clipped rewards.
Optimizer	Adam with learning rate 0.0005, epsilon 1e-08, weight decay 0, and gradient clipping at 10.	Adam with learning rate 6.25e-05, epsilon 0.00015, weight decay 0, and gradient clipping at 10.
Common DQN update	Gamma 0.99; train every 4 environment steps after 5,000 learning-start steps; 1 gradient step per update.	Gamma 0.99; train every 4 environment steps after 20,000 learning-start steps; 1 gradient step per update.
Replay and target defaults	Replay size 100,000; batch size 64; hard target update every 1,000 steps; target tau 1.	Replay size 25,000; batch size 32; hard target update every 8,000 steps; target tau 1.
No-replay override	Replay size 0; batch size 1; learning starts 0; train every 1 step.	Replay size 0; batch size 1; learning starts 0; train every 1 step.
Exploration schedule	Epsilon-greedy from 1 to 0.05 over 100,000 steps.	Epsilon-greedy from 1 to 0.01 over 250,000 steps.

3 Experiment and Discussion

3.1 Lunar Lander

The Lunar Lander results show a clear separation between learning to reach the success threshold once and keeping a usable landing policy until the end of training. Table 3 summarizes the six variants over ten seeds. The success reference is the same as in the method section: an evaluation return of at least 200. The table also reports whether this level was sustained over repeated evaluations, because a single high checkpoint can hide later collapse.

3.1.1 Learning Curves

Figure 4 shows that the replay-buffer variants all learn useful policies, but they do not improve in exactly the same way. The baseline DQN reaches the 200 return threshold earlier on average than the Double and Dueling variants, with a mean first-threshold step of 213k. However, its final mean return is 208.0, below the best final variants. Double DQN and Double + Dueling DQN end at almost the same final mean return, 219.7 and 219.8. This suggests that the larger final gain comes more from stable value updates than from faster early learning.

The no-replay ablation is the main failure case. It has some early positive movement, but the mean curve drops far below the other variants and ends at -260.6. This supports the expectation from the ablation design: for Lunar Lander, recent correlated transitions are not enough to keep a reliable value function, even though the state space is small.

Table 3: Lunar Lander performance, retention, and stability summary over ten seeds.

Variant	Best Eval	Final Eval	Reached 200	Sustained 200	First 200	Final Success	Peak Gap	Largest Drop	Q Spike
DQN	260.2 $\pm$ 17.3	208.0 $\pm$ 73.7	10/10	7/10	213k	75.0%	52.2	251.2	10%
No Target	259.8 $\pm$ 9.4	149.3 $\pm$ 109.9	10/10	0/10	186k	50.0%	110.5	292.2	100%
No Replay	127.6 $\pm$ 53.2	-260.6 $\pm$ 215.7	1/10	0/10	80k	1.5%	388.2	143.3	80%
Double DQN	267.2 $\pm$ 5.2	219.7 $\pm$ 51.7	10/10	5/10	261k	74.0%	47.6	224.1	20%
Dueling DQN	268.3 $\pm$ 10.1	205.5 $\pm$ 68.4	10/10	6/10	255k	69.0%	62.8	261.4	40%
Double + Dueling	258.3 $\pm$ 34.3	219.8 $\pm$ 79.7	9/10	9/10	261k	77.5%	38.4	187.6	40%

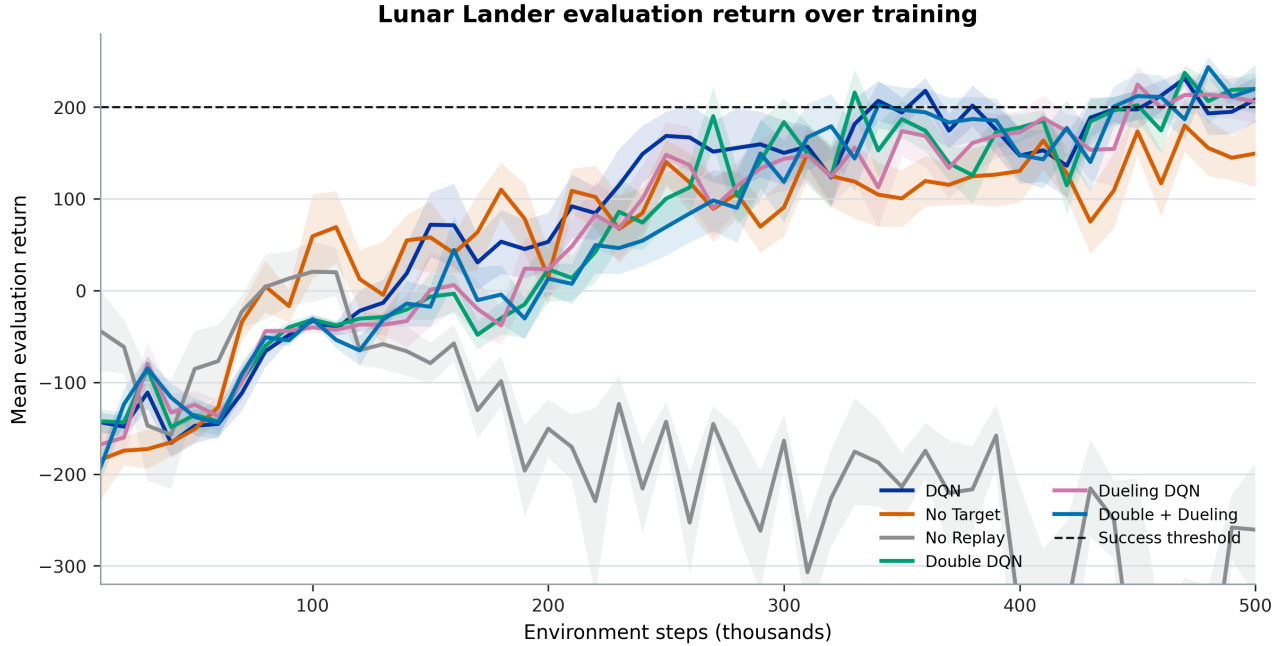


Figure 4: Mean Lunar Lander evaluation return over ten seeds. Shaded bands show the standard error across seeds, and the dashed line marks the return-200 success threshold.

3.1.2 Final Behavior

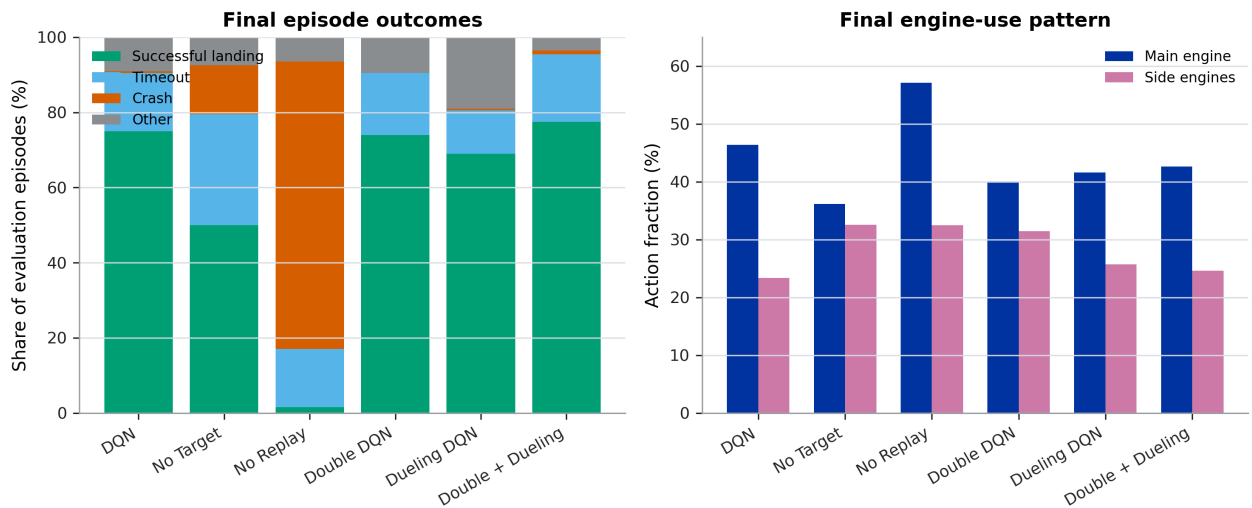


Figure 5: Final Lunar Lander behavior. The left panel compares episode outcomes, and the right panel compares final engine-use fractions.

Figure 5 gives a more concrete view of what the final returns mean. The stronger variants mostly shift from timeout-heavy hovering to completed landings. Double + Dueling DQN has the highest final landing success

rate at 77.5%, followed by DQN at 75.0% and Double DQN at 74.0%. The no-replay variant instead crashes in 76.5% of final evaluation episodes and succeeds in only 1.5%, which explains why its final return is much lower than the others.

The action pattern also changes with better policies. The no-replay policy uses the main engine in 57.1% of final actions, much more than the successful variants. This looks like inefficient vertical control rather than a controlled landing strategy. Double DQN uses the side engines more than the other strong variants, while Double + Dueling DQN gets the best landing rate with a lower side-engine fraction. This suggests that the combined variant does not merely fire more engines; it uses the compact state information more selectively near the landing pad.

3.1.3 Policy Retention

The retention results are the most important extra insight from Lunar Lander. The no-target variant reaches return 200 in all ten seeds and reaches it earlier than the other variants on average, but it sustains the threshold in zero seeds. Its mean peak-to-final gap is 110.5, and its mean largest evaluation drop is 292.2. In plain terms, it can discover a good landing policy, but the policy is not held reliably as training continues.

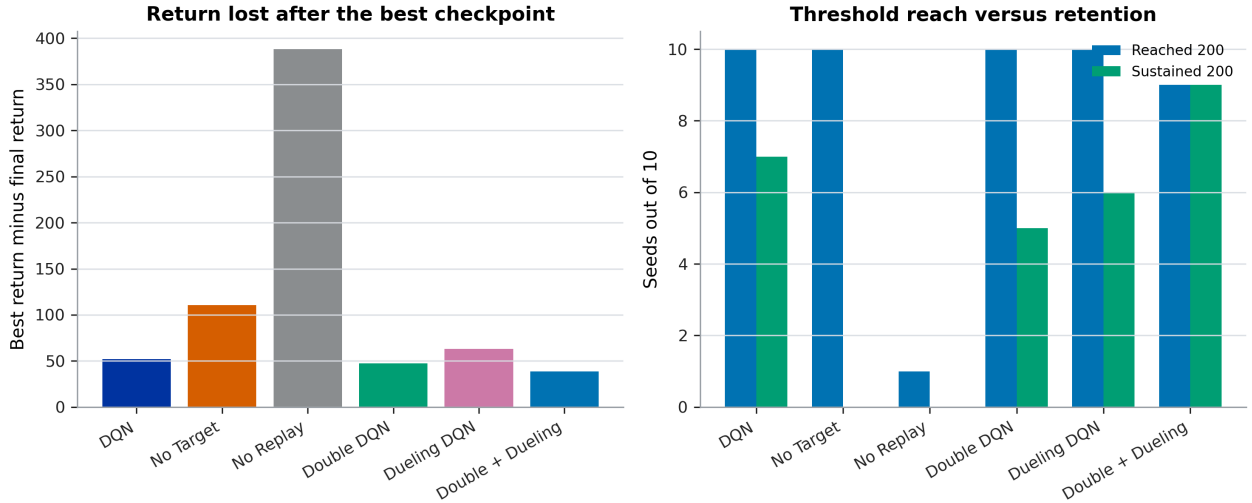


Figure 6: Policy retention after learning. The left panel shows the average drop from each seed’s best evaluation return to its final evaluation return. The right panel compares seeds that reached the threshold with seeds that sustained it.

Double + Dueling DQN shows the opposite pattern. It reaches the threshold in nine seeds rather than all ten, but it also sustains the threshold in nine seeds and has the smallest peak-to-final gap among the high-performing variants, 38.4. Double DQN has the lowest final-return standard deviation among the best final performers, 51.7, so it is the more consistent final-return choice. The combined variant is better when the main concern is holding a successful policy once it appears.

3.1.4 Optimization Stress

Figure 7 explains why the two ablations fail in different ways. The no-replay variant has a much larger value scale than the others, with a mean maximum Q value around 2443.8 and a final TD error around 2.12. This is consistent with unstable bootstrapping from narrow recent experience. The no-target variant has a smaller value scale, but it has a 100% large-Q-spike rate and the highest mean collapse count, 18.8. This matches the retention result: without a delayed target network, learning can make progress, but the target changes too quickly for the policy to stay stable.

Overall, Lunar Lander confirms that replay is essential for this implementation, while the target network is essential for keeping good behavior after it is found. The best final choices are Double DQN and Double + Dueling DQN. Double DQN is the steadier final-return option, and Double + Dueling DQN is the stronger retention option.

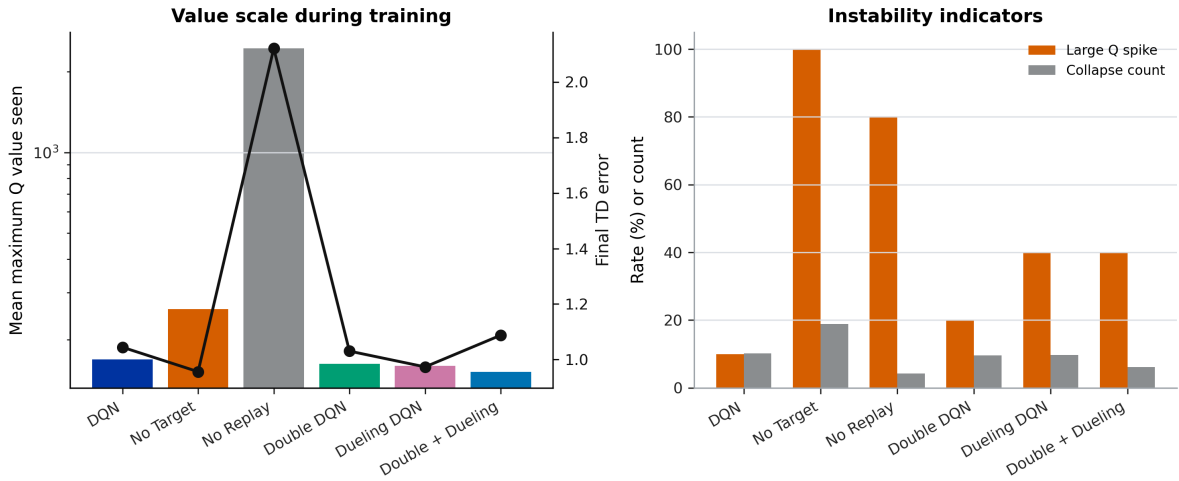


Figure 7: Optimization stress indicators for Lunar Lander. The value-scale panel uses a log axis for maximum Q values and overlays final TD error. The instability panel compares large-Q-spike rate with average collapse count.

3.2 Atari Freeway

The Atari Freeway results test the same DQN variants in a sparse visual task. Table 4 summarizes the six variants over four seeds. The main success reference is a score of at least 15, as in the task-selection table. I also report whether each variant reaches score 22.5, because this separates basic crossing behavior from policies that are closer to the strong final scores.

Table 4: Atari Freeway performance, retention, and final behavior summary over four seeds.

Variant	Best Eval	Final Eval	Reached 15	Sustained 15	First 15	Reached 22.5	Final Zero	Up Action	Peak Gap
DQN	29.1 ± 0.8	28.3 ± 0.7	4/4	4/4	198k	4/4	0.0%	79.8%	0.8
No Target	29.3 ± 0.3	27.2 ± 1.2	4/4	4/4	55k	4/4	0.0%	76.2%	2.1
No Replay	21.6 ± 2.8	5.8 ± 9.7	4/4	1/4	32k	2/4	56.2%	30.0%	15.7
Double DQN	29.7 ± 0.9	29.3 ± 1.1	4/4	4/4	238k	4/4	0.0%	80.0%	0.4
Dueling DQN	29.7 ± 0.5	28.9 ± 0.8	4/4	4/4	245k	4/4	0.0%	77.9%	0.8
Double + Dueling	29.2 ± 0.2	28.3 ± 0.8	4/4	4/4	145k	4/4	0.0%	79.4%	0.9

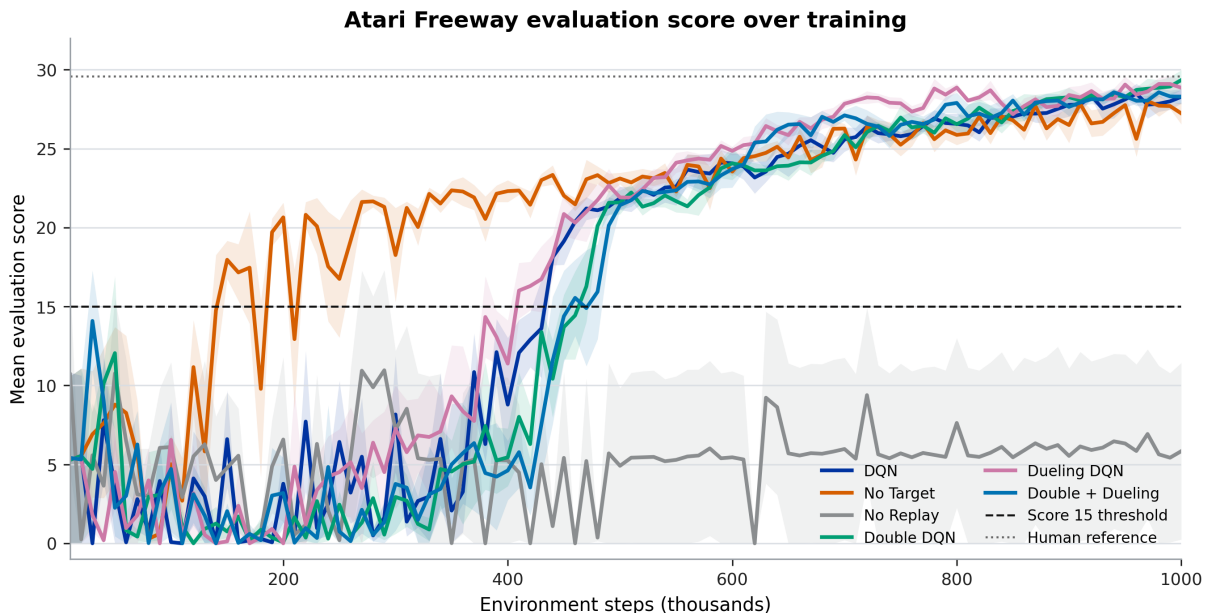


Figure 8: Mean Atari Freeway evaluation score over four seeds. Shaded bands show the standard error across seeds. The dashed line marks the score-15 threshold, and the dotted line marks the human reference score from Mnih et al. [5].

3.2.1 Learning Curves

Figure 8 shows that the replay-buffer variants all learn useful crossing policies and move toward the human reference score of 29.6 from Mnih et al. [5]. Double DQN has the strongest final score, 29.3, and Dueling DQN is close at 28.9. The baseline DQN and Double + Dueling DQN also finish at 28.3, so the main replay-backed variants all end in a similar high-score band.

The learning speed pattern is different from the final-score pattern. The no-target variant reaches score 15 at 55k steps on average, much earlier than DQN, Double DQN, and Dueling DQN. However, it ends below the best final variants at 27.2. This suggests that removing the target network can make the agent find a useful crossing direction quickly, but the same design does not give the best final policy.

The no-replay variant is the main failure case. It reaches score 15 in all four seeds and does so very early, at 32k steps on average, but its final score drops to 5.8 with a large seed-to-seed spread. In Freeway, early reward is therefore not enough evidence of stable learning. A policy can discover upward movement and then lose it later.

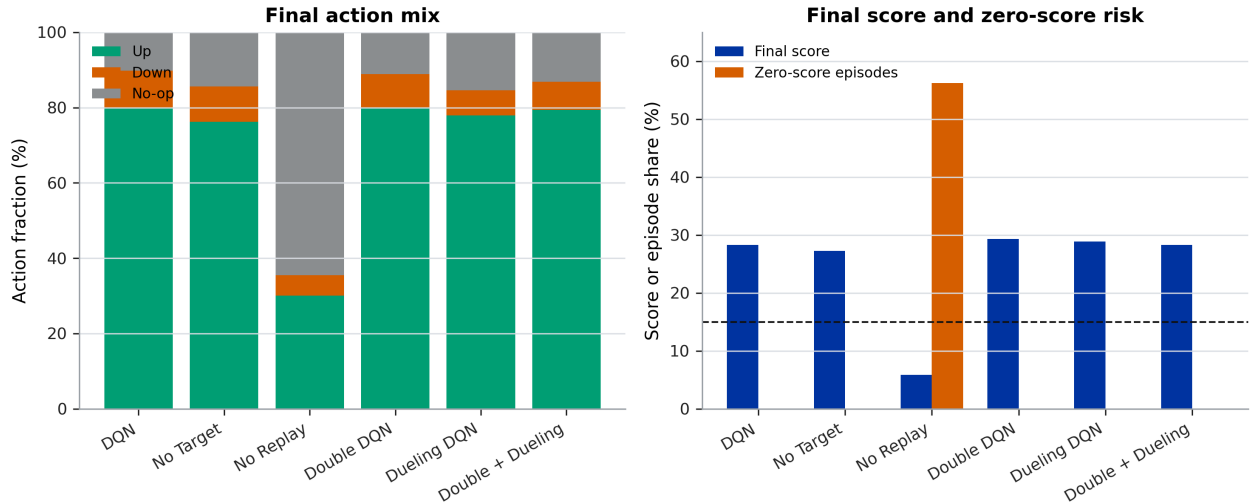


Figure 9: Final Atari Freeway behavior. The left panel compares final action fractions, and the right panel compares final score with zero-score episode rate.

3.2.2 Final Behavior

Figure 9 shows that the successful final policies have a simple action pattern. They mostly choose the upward action, with final up-action fractions between 76.2% and 80.0% for the replay-backed and no-target variants. This matches the task structure: the agent must keep moving upward while traffic interrupts progress.

The no-replay final policy looks different. Its final up-action fraction is only 30.0%, while no-op actions take 64.5% of its final behavior. It also has a 56.2% final zero-score rate. This explains why its final return is much lower than its early threshold result. The failed policy is not only noisy; in most final episodes it stops making enough upward progress to score.

3.2.3 Policy Retention

The retention results are the clearest Freeway insight. The no-replay variant reaches score 15 in all four seeds, but it sustains score 15 in only one seed. It also reaches score 22.5 in only two seeds and has a mean peak-to-final gap of 15.7. This is a stronger version of the retention issue seen in Lunar Lander: finding a good policy once does not mean the agent will keep it.

The replay-backed variants show much better retention. DQN, Double DQN, Dueling DQN, and Double + Dueling DQN all reach and sustain score 15 in all four seeds, and all four variants reach score 22.5 in all four seeds. Double DQN has the smallest peak-to-final gap, 0.4, which matches its best final score. For Freeway, Double DQN is the best final-retention choice.

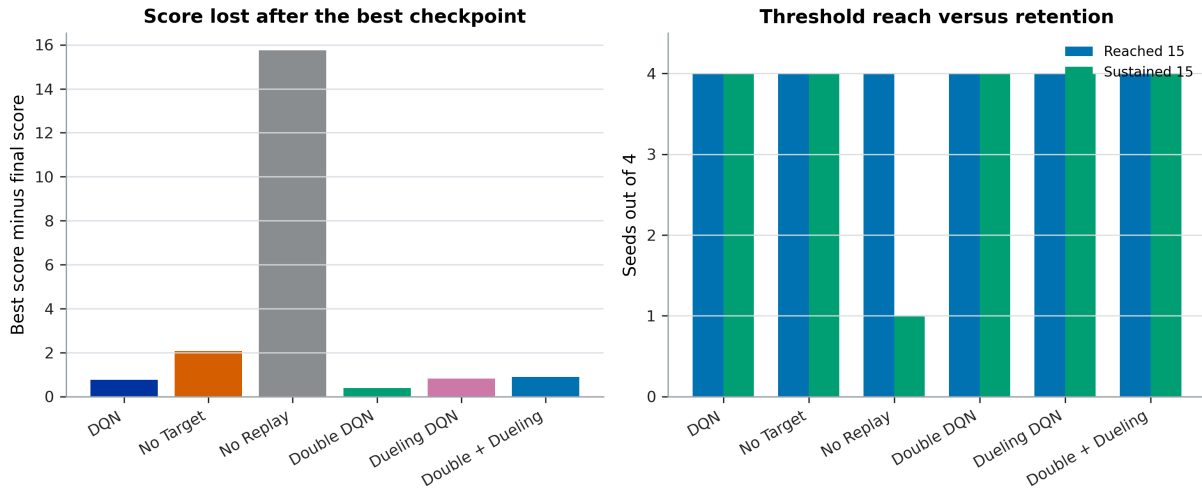


Figure 10: Atari Freeway policy retention after learning. The left panel shows the average drop from each seed’s best evaluation score to its final evaluation score. The right panel compares seeds that reached score 15 with seeds that sustained it.

3.2.4 Optimization Stress

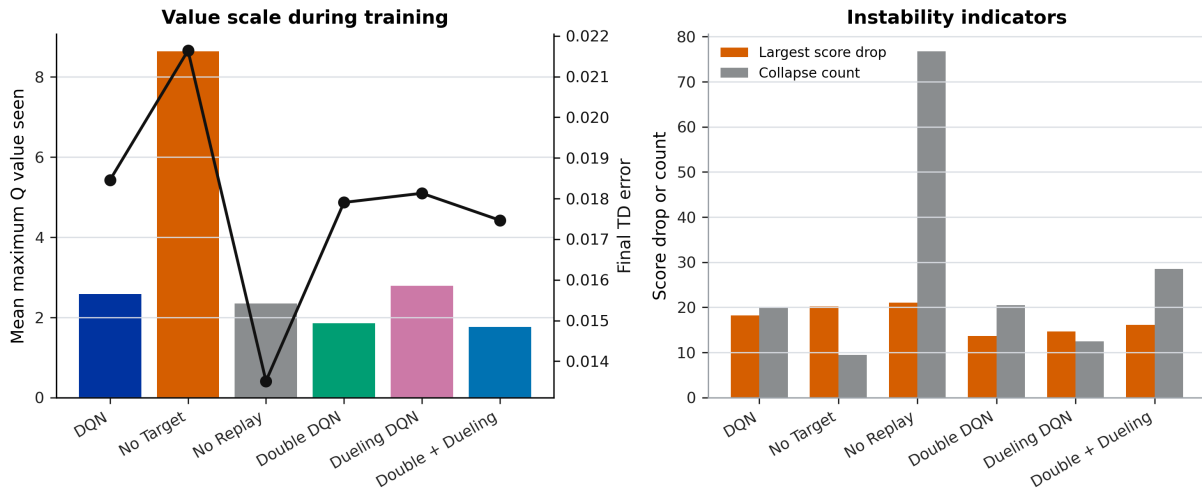


Figure 11: Optimization stress indicators for Atari Freeway. The value-scale panel overlays final TD error, and the instability panel compares largest score drop with average collapse count.

Figure 11 shows that the target-network ablation has a different kind of stress from the replay ablation. The no-target variant has the largest value scale, with a mean maximum Q value of 8.64, while Double DQN and Double + Dueling DQN stay much lower at 1.86 and 1.76. The no-target variant also has the largest final TD error among the high-scoring variants. This supports the same idea as in Lunar Lander: the target network is important for keeping value estimates controlled.

The no-replay variant does not fail through a large Q-value scale. Its mean maximum Q value is 2.35 and its final TD error is small, but it has the highest mean collapse count, 76.8, and the worst final score. This means the loss can look calm after the policy has already become poor. In sparse Freeway, replay is essential because the agent needs to keep useful crossing experience in the update stream instead of learning mainly from recent low-reward behavior.

Overall, Freeway confirms that replay is the most important stabilizer for this implementation. The no-replay agent can discover crossing behavior under this run setting, but it usually cannot keep it. The target network matters more for value-scale control and final quality than for first reaching the threshold. Among the complete variants, Double DQN gives the best final score and the strongest retention.

3.3 Aggregate Analysis

The two task sections show the same six variants from different angles. To compare them directly, Table 5 normalizes each final score by its task threshold: return 200 for Lunar Lander and score 15 for Freeway. A value above 1.0 means the final policy is above the task reference. The same table also compares sustained-threshold rates and the normalized gap between the best checkpoint and the final checkpoint.

Table 5: Cross-task comparison using task-threshold-normalized performance and retention.

Variant	Lunar Final/Thr.	Freeway Final/Thr.	Mean Final/Thr.	Lunar Sustained	Freeway Sustained	Mean Sustained	Mean Peak Gap/Thr.
DQN	1.04	1.89	1.46	70.0%	100.0%	85.0%	0.16
No Target	0.75	1.82	1.28	0.0%	100.0%	50.0%	0.35
No Replay	-1.30	0.39	-0.46	0.0%	25.0%	12.5%	1.50
Double DQN	1.10	1.96	1.53	50.0%	100.0%	75.0%	0.13
Dueling DQN	1.03	1.92	1.48	60.0%	100.0%	80.0%	0.18
Double + Dueling	1.10	1.89	1.49	90.0%	100.0%	95.0%	0.13

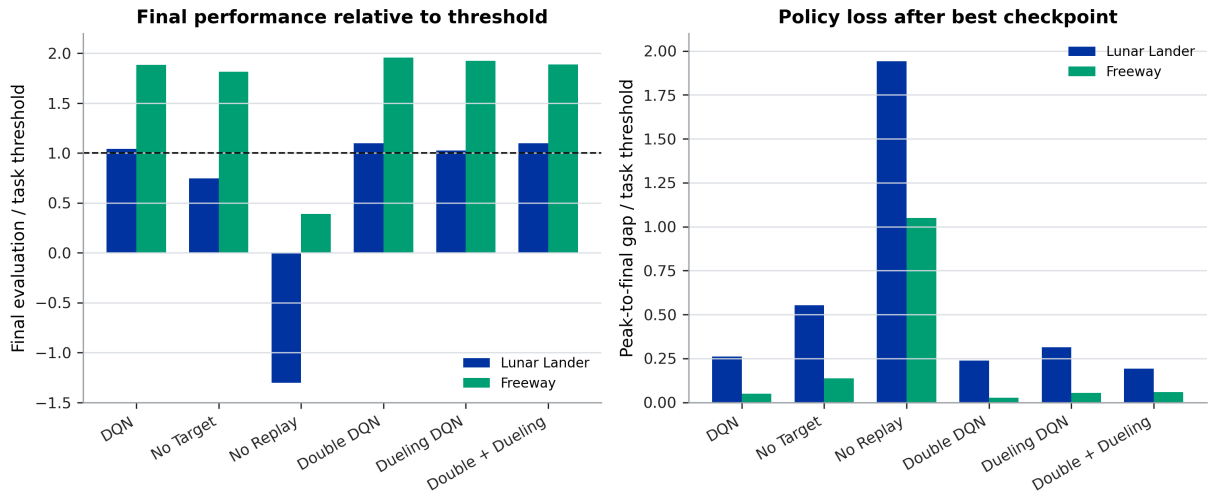


Figure 12: Cross-task aggregate stability. The left panel compares final performance divided by each task threshold. The right panel compares how much performance was lost from the best checkpoint to the final checkpoint, again divided by the task threshold.

The first cross-task insight is that early threshold crossing is a weak signal unless paired with retention. This pattern appears within each task, but becomes clearer in aggregate. The no-target variant reaches the main threshold in every seed for both tasks, yet its mean sustained rate across tasks is only 50.0%. The no-replay variant is weaker: it reaches score 15 in every Freeway seed and briefly reaches return 200 in one Lunar Lander seed, but its mean sustained rate is only 12.5%. In both environments, a policy can discover useful behavior without retaining it.

The second cross-task insight is that the best variant depends on whether the goal is final score or retained success. Double DQN has the best mean final performance after threshold normalization, 1.53, because it combines a strong Lunar Lander final return with the best Freeway final score. Double + Dueling DQN is slightly lower on this final-performance average, 1.49, but has the best mean sustained rate, 95.0%, and nearly the same normalized peak-to-final gap. Thus, Double DQN is the best aggregate final-score choice, whereas Double + Dueling DQN is better for retention.

Figure 12 further shows that the same ablation can fail with different symptoms across tasks. The no-replay variant finishes below the task threshold in both environments and has large peak-to-final loss in both. The no-target variant differs: it remains above the Freeway threshold at the end, but loses much more of its Lunar Lander peak. This supports the broader conclusion that replay is the more basic requirement for learning useful behavior, while the target network is most important for preserving stable value estimates after useful behavior emerges.

A Appendix

A.1 Project Source Bundle

This appendix lists the source code used for this project.

Listing 1: Source code bundle generated by gitingest for this project.

```
1  =====
2  FILE: __init__.py
3  =====
4  """Training utilities for the RL analysis project."""
5
6  from rl_analysis.config import VARIANT_NAMES, build_run_config, get_env_profile, get_variant_spec
7
8  __all__ = [
9      "VARIANT_NAMES",
10     "build_run_config",
11     "get_env_profile",
12     "get_variant_spec",
13 ]
14
15
16
17  =====
18  FILE: agent.py
19  =====
20  """DQN agent implementation."""
21
22  from __future__ import annotations
23
24  import math
25  import time
26  from typing import Any
27
28  import numpy as np
29  import torch
30  from torch import nn
31  from torch.nn import functional as F
32
33  from rl_analysis.config import DQNHypereparameters, OptimizerConfig, VariantSpec
34  from rl_analysis.replay import TransitionBatch
35
36
37  def _as_tensor(array: np.ndarray, device: torch.device, dtype: torch.dtype) -> torch.Tensor:
38      return torch.as_tensor(array, dtype=dtype, device=device)
39
40
41  def _diagnostic_tensor(tensor: torch.Tensor) -> torch.Tensor:
42      tensor = tensor.detach()
43      if tensor.device.type == "mps":
44          return tensor.float().cpu()
45      return tensor
46
47
48  def calculate_dqn_targets(
49      *,
50      online_network: nn.Module,
51      target_network: nn.Module,
52      rewards: torch.Tensor,
53      next_states: torch.Tensor,
54      terminated: torch.Tensor,
55      gamma: float,
56      use_double_dqn: bool,
57  ) -> tuple[torch.Tensor, dict[str, float]]:
58      with torch.no_grad():
59          target_next_q = target_network(next_states)
60          vanilla_max_target_q = target_next_q.max(dim=1).values
61          if use_double_dqn:
62              online_next_q = online_network(next_states)
63              next_actions = online_next_q.argmax(dim=1, keepdim=True)
64              selected_next_q = target_next_q.gather(1, next_actions).squeeze(1)
65              online_selected_action_q = online_next_q.gather(1, next_actions).squeeze(1)
```

```

66     else:
67         online_next_q = online_network(next_states)
68         selected_next_q = vanilla_max_target_q
69         online_selected_action_q = online_next_q.max(dim=1).values
70
71     non_terminal = 1.0 - terminated.float()
72     targets = rewards + gamma * non_terminal * selected_next_q
73     diagnostics = {
74         "online_selected_action_q_mean": float(online_selected_action_q.mean().item()),
75         "target_evaluated_selected_action_q_mean": float(selected_next_q.mean().item()),
76         "target_evaluated_selected_action_q_std": float(selected_next_q.std(unbiased=False).item()),
77         "target_evaluated_selected_action_q_min": float(selected_next_q.min().item()),
78         "target_evaluated_selected_action_q_max": float(selected_next_q.max().item()),
79         "vanilla_max_target_q_mean_proxy": float(vanilla_max_target_q.mean().item()),
80         "double_dqn_target_q_mean": float(selected_next_q.mean().item()) if use_double_dqn else None,
81         "estimated_overestimation_reduction": float((vanilla_max_target_q - selected_next_q).mean().item())
82         if use_double_dqn
83         else None,
84     }
85     return targets, diagnostics
86
87
88 class DQNAgent:
89     def __init__(
90         self,
91         *,
92         online_network: nn.Module,
93         target_network: nn.Module | None,
94         variant: VariantSpec,
95         dqn_config: DQNHypereparameters,
96         optimizer_config: OptimizerConfig,
97         device: torch.device,
98     ):
99         self.online_network = online_network.to(device)
100         self.target_network = target_network.to(device) if target_network is not None else None
101         self.variant = variant
102         self.dqn_config = dqn_config
103         self.optimizer_config = optimizer_config
104         self.device = device
105         self.optimizer = torch.optim.Adam(
106             self.online_network.parameters(),
107             lr=optimizer_config.learning_rate,
108             eps=optimizer_config.epsilon,
109             weight_decay=optimizer_config.weight_decay,
110         )
111         self.update_index = 0
112         self.target_update_count = 0
113         self.last_target_update_step = 0
114         if self.target_network is not None:
115             self.hard_update_target()
116
117     def select_action(
118         self,
119         observation: np.ndarray,
120         epsilon: float,
121         rng: np.random.Generator,
122         random_action_probabilities: np.ndarray | None = None,
123     ) -> int:
124         if rng.random() < epsilon:
125             if random_action_probabilities is not None:
126                 return int(rng.choice(self.action_dim, p=random_action_probabilities))
127             return int(rng.integers(self.action_dim))
128         obs_tensor = torch.as_tensor(observation, dtype=torch.float32, device=self.device).unsqueeze(0)
129         with torch.no_grad():
130             q_values = self.online_network(obs_tensor)
131         return int(torch.argmax(q_values, dim=1).item())
132
133     @property
134     def action_dim(self) -> int:
135         return int(getattr(self.online_network, "action_dim"))
136
137     def hard_update_target(self, global_env_step: int = 0) -> None:
138         if self.target_network is None:
139             return

```

```

40     self.target_network.load_state_dict(self.online_network.state_dict())
41     self.target_update_count += 1
42     self.last_target_update_step = global_env_step
43
44     def maybe_update_target(self, global_env_step: int) -> bool:
45         if self.target_network is None:
46             return False
47         frequency = self.dqn_config.target_update_frequency_env_steps
48         if frequency <= 0 or global_env_step <= 0 or global_env_step % frequency != 0:
49             return False
50         tau = self.dqn_config.target_update_tau
51         if tau >= 1.0:
52             self.hard_update_target(global_env_step)
53         else:
54             with torch.no_grad():
55                 for target_param, online_param in zip(
56                     self.target_network.parameters(), self.online_network.parameters(), strict=True
57                 ):
58                     target_param.data.mul_(1.0 - tau).add_(online_param.data, alpha=tau)
59                 self.target_update_count += 1
60                 self.last_target_update_step = global_env_step
61         return True
62
63     def target_parameter_distance(self) -> tuple[float | None, float | None]:
64         if self.target_network is None:
65             return None, None
66         sq_sum = 0.0
67         count = 0
68         with torch.no_grad():
69             for online_param, target_param in zip(
70                 self.online_network.parameters(), self.target_network.parameters(), strict=True
71             ):
72                 diff = (online_param - target_param).detach()
73                 sq_sum += float(torch.sum(diff * diff).item())
74                 count += diff.numel()
75         l2 = math.sqrt(sq_sum)
76         mean_per_param = l2 / max(1, count)
77         return l2, mean_per_param
78
79     def update(self, batch: TransitionBatch, global_env_step: int) -> dict[str, Any]:
80         start = time.perf_counter()
81         states = _as_tensor(batch.states, self.device, torch.float32)
82         actions = _as_tensor(batch.actions, self.device, torch.int64)
83         rewards = _as_tensor(batch.rewards, self.device, torch.float32)
84         next_states = _as_tensor(batch.next_states, self.device, torch.float32)
85         terminated = _as_tensor(batch.terminated.astype(np.float32), self.device, torch.float32)
86
87         target_network = self.target_network if self.target_network is not None else self.online_network
88         targets, double_dqn_diag = calculate_dqn_targets(
89             online_network=self.online_network,
90             target_network=target_network,
91             rewards=rewards,
92             next_states=next_states,
93             terminated=terminated,
94             gamma=self.dqn_config.gamma,
95             use_double_dqn=self.variant.use_double_dqn,
96         )
97
98         q_values, streams = self.online_network(states, return_streams=True)
99         chosen_q = q_values.gather(1, actions.unsqueeze(1)).squeeze(1)
100         td_errors = targets - chosen_q
101         per_sample_loss = F.smooth_l1_loss(chosen_q, targets, reduction="none")
102         loss = per_sample_loss.mean()
103
104         self.optimizer.zero_grad(set_to_none=True)
105         loss.backward()
106         grad_norm_before = torch.nn.utils.clip_grad_norm_(
107             self.online_network.parameters(), self.dqn_config.max_grad_norm
108         )
109         self.optimizer.step()
110         self.update_index += 1
111
112         grad_norm_after = self._grad_norm()
113         param_norm = self._param_norm()

```

```

114 top2 = torch.topk(q_values.detach(), k=min(2, q_values.shape[1]), dim=1).values
115 action_gap = top2[:, 0] - top2[:, 1] if top2.shape[1] > 1 else torch.zeros_like(top2[:, 0])
116 target_param_l2, target_param_l2_mean = self.target_parameter_distance()
117
118 td_abs = torch.abs(td_errors.detach())
119 td_abs_stats = _diagnostic_tensor(td_abs)
120 target_q_mean = double_dqn_diag["target_evaluated_selected_action_q_mean"]
121 online_q_max_mean = float(q_values.detach().max(dim=1).values.mean().item())
122 metrics = {
123     "global_env_step": global_env_step,
124     "update_index": self.update_index,
125     "loss": {
126         "td_loss_mean": float(loss.detach().item()),
127         "td_loss_std": float(per_sample_loss.detach().std(unbiased=False).item()),
128         "td_error_mean": float(td_errors.detach().mean().item()),
129         "td_error_abs_mean": float(td_abs_stats.mean().item()),
130         "td_error_abs_median": float(torch.median(td_abs_stats).item()),
131         "td_error_abs_p90": float(torch.quantile(td_abs_stats, 0.90).item()),
132         "td_error_abs_p95": float(torch.quantile(td_abs_stats, 0.95).item()),
133         "td_error_abs_p99": float(torch.quantile(td_abs_stats, 0.99).item()),
134     },
135     "q_values": {
136         "online_q_mean": float(q_values.detach().mean().item()),
137         "online_q_std": float(q_values.detach().std(unbiased=False).item()),
138         "online_q_min": float(q_values.detach().min().item()),
139         "online_q_max": float(q_values.detach().max().item()),
140         "online_q_max_mean": online_q_max_mean,
141         "target_q_mean": target_q_mean,
142         "target_q_std": double_dqn_diag["target_evaluated_selected_action_q_std"],
143         "target_q_min": double_dqn_diag["target_evaluated_selected_action_q_min"],
144         "target_q_max": double_dqn_diag["target_evaluated_selected_action_q_max"],
145         "q_overestimation_proxy": float(online_q_max_mean - target_q_mean),
146         "action_gap_mean": float(action_gap.mean().item()),
147         "action_gap_std": float(action_gap.std(unbiased=False).item()),
148     },
149     "targets": {
150         "bellman_target_mean": float(targets.detach().mean().item()),
151         "bellman_target_std": float(targets.detach().std(unbiased=False).item()),
152         "bellman_target_min": float(targets.detach().min().item()),
153         "bellman_target_max": float(targets.detach().max().item()),
154         "reward_mean": float(rewards.mean().item()),
155         "reward_std": float(rewards.std(unbiased=False).item()),
156         "done_fraction": float(np.mean(batch.done_for_logging)),
157     },
158     "optimization": {
159         "learning_rate": self.optimizer_config.learning_rate,
160         "grad_norm": float(grad_norm_after),
161         "grad_norm_before_clip": float(grad_norm_before.item()),
162         "grad_norm_after_clip": float(grad_norm_after),
163         "grad_norm_clipped": bool(float(grad_norm_before.item()) > self.dqn_config.max_grad_norm),
164         "param_norm": float(param_norm),
165         "update_wall_time_sec": time.perf_counter() - start,
166     },
167     "target_network": {
168         "enabled": self.target_network is not None,
169         "target_update_count": self.target_update_count if self.target_network is not None else None,
170         "steps_since_target_update": global_env_step - self.last_target_update_step
171         if self.target_network is not None
172         else None,
173         "target_update_frequency_env_steps": self.dqn_config.target_update_frequency_env_steps
174         if self.target_network is not None
175         else None,
176         "target_update_tau": self.dqn_config.target_update_tau if self.target_network is not None else None,
177         "online_target_param_l2": target_param_l2,
178         "online_target_param_l2_mean_per_param": target_param_l2_mean,
179     },
180     "double_dqn": {
181         "enabled": self.variant.use_double_dqn,
182         **double_dqn_diag,
183     },
184     "dueling": self._dueling_metrics(streams, action_gap),
185     "failure_diagnostics": self._failure_diagnostics(loss, q_values, grad_norm_before),
186 }
187 return metrics

```

```

288
289 def _dueling_metrics(self, streams: dict[str, torch.Tensor | None], action_gap: torch.Tensor) -> dict[str, Any]:
290     if not self.variant.use_dueling_network:
291         return {
292             "enabled": False,
293             "value_stream_mean": None,
294             "value_stream_std": None,
295             "advantage_stream_mean": None,
296             "advantage_stream_std": None,
297             "advantage_abs_mean": None,
298             "advantage_max_mean": None,
299             "advantage_min_mean": None,
300             "action_gap_mean": None,
301             "action_gap_std": None,
302         }
303     value = streams["value"].detach()
304     advantage = streams["advantage"].detach()
305     return {
306         "enabled": True,
307         "value_stream_mean": float(value.mean().item()),
308         "value_stream_std": float(value.std(unbiased=False).item()),
309         "advantage_stream_mean": float(advantage.mean().item()),
310         "advantage_stream_std": float(advantage.std(unbiased=False).item()),
311         "advantage_abs_mean": float(torch.abs(advantage).mean().item()),
312         "advantage_max_mean": float(advantage.max(dim=1).values.mean().item()),
313         "advantage_min_mean": float(advantage.min(dim=1).values.mean().item()),
314         "action_gap_mean": float(action_gap.mean().item()),
315         "action_gap_std": float(action_gap.std(unbiased=False).item()),
316     }
317
318 def _grad_norm(self) -> float:
319     sq_sum = 0.0
320     for param in self.online_network.parameters():
321         if param.grad is not None:
322             sq_sum += float(torch.sum(param.grad.detach() * param.grad.detach()).item())
323     return math.sqrt(sq_sum)
324
325 def _param_norm(self) -> float:
326     sq_sum = 0.0
327     with torch.no_grad():
328         for param in self.online_network.parameters():
329             sq_sum += float(torch.sum(param * param).item())
330     return math.sqrt(sq_sum)
331
332 def _failure_diagnostics(
333     self, loss: torch.Tensor, q_values: torch.Tensor, grad_norm_before: torch.Tensor
334 ) -> dict[str, Any]:
335     q_abs_max = float(torch.max(torch.abs(q_values.detach()))).item()
336     loss_value = float(loss.detach().item())
337     grad_value = float(grad_norm_before.item())
338     return {
339         "nan_detected": bool(torch.isnan(loss).item() or torch.isnan(q_values).any().item()),
340         "inf_detected": bool(torch.isinf(loss).item() or torch.isinf(q_values).any().item()),
341         "q_value_explosion": bool(q_abs_max > 1000.0),
342         "q_value_explosion_threshold": 1000.0,
343         "loss_explosion": bool(loss_value > 1000.0),
344         "loss_explosion_threshold": 1000.0,
345         "gradient_explosion": bool(grad_value > 100.0),
346         "gradient_explosion_threshold": 100.0,
347         "early_terminated": False,
348         "early_terminated_reason": None,
349     }
350
351
352
353 =====
354 FILE: aggregate.py
355 =====
356 """Aggregate completed experiment runs."""
357
358 from __future__ import annotations
359
360 import argparse
361 from collections import defaultdict

```



```

362 from pathlib import Path
363 from typing import Any
364
365 import matplotlib
366
367 matplotlib.use("Agg")
368 import matplotlib.pyplot as plt
369 import numpy as np
370
371 from rl_analysis.config import VARIANT_NAMES, get_env_profile
372 from rl_analysis.utils import read_jsonl, write_json
373
374
375 def parse_args() -> argparse.Namespace:
376     parser = argparse.ArgumentParser(description="Aggregate DQN experiment summaries.")
377     parser.add_argument("--env", default="LunarLander-v3")
378     parser.add_argument("--output-root", default="output")
379     return parser.parse_args()
380
381
382 def _latest_seed_runs(variant_dir: Path) -> list[Path]:
383     runs = []
384     for seed_dir in sorted(variant_dir.glob("seed_*")):
385         candidates = [
386             run_dir
387             for run_dir in sorted(seed_dir.iterdir())
388             if run_dir.is_dir() and (run_dir / "summary.json").exists()
389         ]
390         if not candidates:
391             continue
392         runs.append(max(candidates, key=lambda path: path.stat().st_mtime))
393     return runs
394
395
396 def _load_summary(path: Path) -> dict[str, Any]:
397     import json
398
399     return json.loads(path.read_text(encoding="utf-8"))
400
401
402 def _stats(values: list[float]) -> dict[str, float | None]:
403     arr = np.asarray(values, dtype=np.float64)
404     if arr.size == 0:
405         return {"mean": None, "std": None, "min": None, "max": None, "median": None}
406     return {
407         "mean": float(np.mean(arr)),
408         "std": float(np.std(arr)),
409         "min": float(np.min(arr)),
410         "max": float(np.max(arr)),
411         "median": float(np.median(arr)),
412     }
413
414
415 def aggregate_variant(env_slug: str, variant: str, output_root: Path) -> dict[str, Any] | None:
416     variant_dir = output_root / env_slug / variant
417     run_dirs = _latest_seed_runs(variant_dir)
418     summaries = [_load_summary(run_dir / "summary.json") for run_dir in run_dirs]
419     summaries = [summary for summary in summaries if summary.get("training_completed")]
420     if not summaries:
421         return None
422
423     seeds = [int(summary["seed"]) for summary in summaries]
424     final_eval = [
425         summary["performance"]["final_eval_return_mean"]
426         for summary in summaries
427         if summary["performance"]["final_eval_return_mean"] is not None
428     ]
429     best_eval = [
430         summary["performance"]["best_eval_return_mean"]
431         for summary in summaries
432         if summary["performance"]["best_eval_return_mean"] is not None
433     ]
434     auc = [
435         summary["performance"]["area_under_eval_curve"]

```

```

436     for summary in summaries
437     if summary["performance"]["area_under_eval_curve"] is not None
438 ]
439 reached_steps = [
440     summary["sample_efficiency"]["first_step_reaching_threshold"]
441     for summary in summaries
442     if summary["sample_efficiency"]["first_step_reaching_threshold"] is not None
443 ]
444 collapse_counts = [summary["stability"]["catastrophic_collapse_count"] for summary in summaries]
445 largest_drops = [
446     summary["stability"]["largest_eval_drop"]
447     for summary in summaries
448     if summary["stability"]["largest_eval_drop"] is not None
449 ]
450
451 aggregate = {
452     "env_id": summaries[0]["env_id"],
453     "variant": variant,
454     "num_seeds": len(seeds),
455     "seeds": sorted(seeds),
456     "final_eval_return": _stats(final_eval),
457     "best_eval_return": _stats(best_eval),
458     "area_under_eval_curve": _stats(auc),
459     "steps_to_threshold": {
460         "threshold": summaries[0]["sample_efficiency"]["threshold"],
461         **_stats([float(x) for x in reached_steps]),
462         "success_rate": float(len(reached_steps) / len(summaries)),
463         "num_seeds_reached": len(reached_steps),
464     },
465     "stability": {
466         "catastrophic_collapse_count_mean": float(np.mean(collapse_counts)) if collapse_counts else None,
467         "largest_eval_drop_mean": float(np.mean(largest_drops)) if largest_drops else None,
468         "eval_return_across_seed_std_mean": float(np.std(final_eval)) if final_eval else None,
469     },
470     "failure_rate": {
471         "nan_rate": float(np.mean([summary["failure_diagnostics"]["nan_detected"] for summary in summaries])),
472         "q_value_explosion_rate": float(
473             np.mean([summary["failure_diagnostics"]["q_value_explosion"] for summary in summaries])
474         ),
475         "early_termination_rate": float(
476             np.mean([summary["failure_diagnostics"]["early_terminated"] for summary in summaries])
477         ),
478     },
479 }
480 if summaries[0]["env_id"] == "ALE/Freeway-v5":
481     aggregate["freeway"] = _aggregate_freeway(summaries)
482 write_json(variant_dir / "aggregate_summary.json", aggregate)
483 return aggregate
484
485
486 def _aggregate_freeway(summaries: list[dict[str, Any]]) -> dict[str, Any]:
487     final_zero_score_rates = [
488         summary.get("freeway", {}).get("final_zero_score_rate")
489         for summary in summaries
490         if summary.get("freeway", {}).get("final_zero_score_rate") is not None
491     ]
492     thresholds = summaries[0].get("freeway", {}).get("score_thresholds", [])
493     threshold_success_rates = {}
494     threshold_first_steps = {}
495     for threshold in thresholds:
496         key = str(threshold)
497         reached = [
498             summary.get("freeway", {}).get("first_step_reaching_score_thresholds", {}).get(key)
499             for summary in summaries
500         ]
501         reached_steps = [float(step) for step in reached if step is not None]
502         threshold_success_rates[key] = float(len(reached_steps) / len(summaries))
503         threshold_first_steps[key] = _stats(reached_steps)
504     return {
505         "final_zero_score_rate": _stats([float(value) for value in final_zero_score_rates]),
506         "score_threshold_success_rates": threshold_success_rates,
507         "score_threshold_first_steps": threshold_first_steps,
508         "reference_random_score": summaries[0].get("freeway", {}).get("reference_random_score"),
509         "reference_human_score": summaries[0].get("freeway", {}).get("reference_human_score"),

```

```

310     "reference_dqn_score": summaries[0].get("freeway", {}).get("reference_dqn_score"),
311 }
312
313
314 def plot_learning_curves(env_slug: str, output_root: Path) -> None:
315     plot_dir = output_root / env_slug / "plots"
316     plot_dir.mkdir(parents=True, exist_ok=True)
317     plotted_any = False
318     plt.figure(figsize=(8, 5))
319     for variant in VARIANT_NAMES:
320         variant_dir = output_root / env_slug / variant
321         run_dirs = _latest_seed_runs(variant_dir)
322         curves: dict[int, list[float]] = defaultdict(list)
323         for run_dir in run_dirs:
324             for row in read_jsonl(run_dir / "eval_metrics.jsonl"):
325                 if row.get("return_mean") is not None:
326                     curves[int(row["global_env_step"])] .append(float(row["return_mean"]))
327         if not curves:
328             continue
329         steps = sorted(curves)
330         means = [float(np.mean(curves[step])) for step in steps]
331         plt.plot(steps, means, label=variant)
332         plotted_any = True
333     if plotted_any:
334         plt.xlabel("Environment steps")
335         plt.ylabel("Mean evaluation return")
336         plt.title(f"{env_slug} DQN ablations")
337         plt.legend()
338         plt.tight_layout()
339         plt.savefig(plot_dir / "eval_learning_curves.png", dpi=160)
340     plt.close()
341
342
343 def main() -> None:
344     args = parse_args()
345     profile = get_env_profile(args.env)
346     output_root = Path(args.output_root)
347     written = []
348     for variant in VARIANT_NAMES:
349         aggregate = aggregate_variant(profile.slug, variant, output_root)
350         if aggregate is not None:
351             written.append(variant)
352     plot_learning_curves(profile.slug, output_root)
353     if written:
354         print(f"Aggregated variants: {' , '.join(written)}")
355     else:
356         print(f"No completed runs found under {output_root / profile.slug}")
357
358
359 if __name__ == "__main__":
360     main()
361
362
363
364 =====
365 FILE: config.py
366 =====
367 """Dataclass configuration for DQN experiments."""
368
369 from __future__ import annotations
370
371 from dataclasses import dataclass, field, replace
372 from datetime import datetime, timezone
373 from pathlib import Path
374 from typing import Any
375
376
377 def _utc_timestamp() -> str:
378     return datetime.now(timezone.utc).strftime("%Y%m%d_%H%M%S")
379
380
381 @dataclass(frozen=True)
382 class VariantSpec:
383     name: str

```

```

384     label: str
385     use_target_network: bool
386     use_replay_buffer: bool
387     use_double_dqn: bool
388     use_dueling_network: bool
389     use_prioritized_replay: bool = False
390     use_noisy_network: bool = False
391     use_reward_clipping: bool = False
392
393
394     VARIANTS: dict[str, VariantSpec] = {
395         "dqn": VariantSpec(
396             name="dqn",
397             label="DQN",
398             use_target_network=True,
399             use_replay_buffer=True,
400             use_double_dqn=False,
401             use_dueling_network=False,
402         ),
403         "dqn_no_target": VariantSpec(
404             name="dqn_no_target",
405             label="DQN without target network",
406             use_target_network=False,
407             use_replay_buffer=True,
408             use_double_dqn=False,
409             use_dueling_network=False,
410         ),
411         "dqn_no_replay": VariantSpec(
412             name="dqn_no_replay",
413             label="DQN without replay buffer",
414             use_target_network=True,
415             use_replay_buffer=False,
416             use_double_dqn=False,
417             use_dueling_network=False,
418         ),
419         "double_dqn": VariantSpec(
420             name="double_dqn",
421             label="Double DQN",
422             use_target_network=True,
423             use_replay_buffer=True,
424             use_double_dqn=True,
425             use_dueling_network=False,
426         ),
427         "dueling_dqn": VariantSpec(
428             name="dueling_dqn",
429             label="Dueling DQN",
430             use_target_network=True,
431             use_replay_buffer=True,
432             use_double_dqn=False,
433             use_dueling_network=True,
434         ),
435         "double_dueling_dqn": VariantSpec(
436             name="double_dueling_dqn",
437             label="Double DQN + Dueling DQN",
438             use_target_network=True,
439             use_replay_buffer=True,
440             use_double_dqn=True,
441             use_dueling_network=True,
442         ),
443     }
444
445     VARIANT_NAMES = tuple(VARIANTS.keys())
446
447
448     @dataclass(frozen=True)
449     class EnvironmentProfile:
450         env_id: str
451         slug: str
452         observation_type: str
453         action_space_type: str
454         network_architecture: str
455         success_threshold: float | None
456         reward_clipping: bool = False
457         raw_frame_skip: int | None = None

```

```

658 repeat_action_probability: float | None = None
659 full_action_space: bool | None = None
660 frame_skip: int | None = None
661 frame_stack: int | None = None
662 grayscale: bool | None = None
663 resize_shape: tuple[int, int] | None = None
664 noop_max: int | None = None
665 terminal_on_life_loss: bool | None = None
666 max_episode_steps: int | None = None
667 score_thresholds: tuple[float, ...] = ()
668 reference_random_score: float | None = None
669 reference_human_score: float | None = None
670 reference_dqn_score: float | None = None
671
672
673 ENV_PROFILES: dict[str, EnvironmentProfile] = {
674     "LunarLander-v3": EnvironmentProfile(
675         env_id="LunarLander-v3",
676         slug="lunarlander_v3",
677         observation_type="vector",
678         action_space_type="discrete",
679         network_architecture="mlp",
680         success_threshold=200.0,
681         reward_clipping=False,
682     ),
683     "ALE/Freeway-v5": EnvironmentProfile(
684         env_id="ALE/Freeway-v5",
685         slug="freeway",
686         observation_type="image",
687         action_space_type="discrete",
688         network_architecture="cnn",
689         success_threshold=15.0,
690         reward_clipping=True,
691         raw_frame_skip=1,
692         repeat_action_probability=0.25,
693         full_action_space=False,
694         frame_skip=4,
695         frame_stack=4,
696         grayscale=True,
697         resize_shape=(84, 84),
698         noop_max=30,
699         terminal_on_life_loss=False,
700         score_thresholds=(5.0, 10.0, 15.0, 22.5),
701         reference_random_score=0.0,
702         reference_human_score=29.6,
703         reference_dqn_score=30.3,
704     ),
705 }
706
707 ENV_ALIASES = {
708     "lunarlander": "LunarLander-v3",
709     "lunarlander-v3": "LunarLander-v3",
710     "lunarlander_v3": "LunarLander-v3",
711     "freeway": "ALE/Freeway-v5",
712     "atari_freeway": "ALE/Freeway-v5",
713     "ale/freeway-v5": "ALE/Freeway-v5",
714 }
715
716
717 @dataclass
718 class TrainingConfig:
719     total_env_steps: int = 500_000
720     max_episodes: int | None = None
721     num_seeds: int = 5
722     eval_frequency_env_steps: int = 10_000
723     checkpoint_frequency_env_steps: int = 50_000
724     num_eval_episodes: int = 20
725     update_log_frequency: int = 100
726     system_metrics_frequency_env_steps: int = 10_000
727     sustained_threshold_window: int = 5
728     device: str = "auto"
729
730
731 @dataclass

```

```

732 class OptimizerConfig:
733     name: str = "Adam"
734     learning_rate: float = 5e-4
735     epsilon: float = 1e-8
736     weight_decay: float = 0.0
737     gradient_clip_norm: float = 10.0
738
739
740 @dataclass
741 class DQNHypersparameters:
742     gamma: float = 0.99
743     batch_size: int = 64
744     replay_buffer_size: int = 100_000
745     learning_starts: int = 5_000
746     train_frequency_env_steps: int = 4
747     gradient_steps_per_train: int = 1
748     target_update_frequency_env_steps: int = 1_000
749     target_update_tau: float = 1.0
750     max_grad_norm: float = 10.0
751
752
753 @dataclass
754 class ExplorationConfig:
755     type: str = "epsilon_greedy"
756     epsilon_start: float = 1.0
757     epsilon_final: float = 0.05
758     epsilon_decay_env_steps: int = 100_000
759     eval_epsilon: float = 0.001
760     freeway_up_bias: float = 0.0
761
762     def __post_init__(self) -> None:
763         if not 0.0 <= self.freeway_up_bias <= 1.0:
764             raise ValueError("freeway_up_bias must be between 0.0 and 1.0.")
765
766     def epsilon_at(self, env_step: int) -> float:
767         if self.epsilon_decay_env_steps <= 0:
768             return self.epsilon_final
769         fraction = min(1.0, max(0.0, env_step / self.epsilon_decay_env_steps))
770         return self.epsilon_start + fraction * (self.epsilon_final - self.epsilon_start)
771
772
773 @dataclass
774 class NetworkConfig:
775     architecture: str = "mlp"
776     hidden_layers: list[int] = field(default_factory=lambda: [128, 128])
777     activation: str = "relu"
778     dueling_aggregation: str | None = None
779     conv_layers: list[dict[str, int]] = field(
780         default_factory=lambda: [
781             {"out_channels": 32, "kernel_size": 8, "stride": 4},
782             {"out_channels": 64, "kernel_size": 4, "stride": 2},
783             {"out_channels": 64, "kernel_size": 3, "stride": 1},
784         ]
785     )
786     fully_connected_layers: list[int] = field(default_factory=lambda: [512])
787     num_parameters: int | None = None
788     initializer: str = "orthogonal"
789
790
791 @dataclass
792 class RunConfig:
793     run_id: str
794     experiment_group: str
795     env: EnvironmentProfile
796     variant: VariantSpec
797     seed: int
798     output_root: Path
799     run_dir: Path
800     datetime_start: str
801     datetime_end: str | None
802     status: str
803     training: TrainingConfig
804     optimizer: OptimizerConfig
805     dqn: DQNHypersparameters

```

```

$06     exploration: ExplorationConfig
$07     network: NetworkConfig
$08
$09
$10 def get_variant_spec(name: str) -> VariantSpec:
$11     key = name.strip().lower()
$12     if key not in VARIANTS:
$13         allowed = ", ".join(VARIANT_NAMES)
$14         raise ValueError(f"Unknown variant {name!r}. Expected one of: {allowed}.")
$15     return VARIANTS[key]
$16
$17
$18 def get_env_profile(env_id: str) -> EnvironmentProfile:
$19     normalized = env_id.strip()
$20     canonical = ENV_ALIASES.get(normalized.lower(), normalized)
$21     if canonical not in ENV_PROFILES:
$22         allowed = ", ".join(ENV_PROFILES)
$23         raise ValueError(f"Unknown environment {env_id!r}. Expected one of: {allowed}.")
$24     return ENV_PROFILES[canonical]
$25
$26
$27 def default_training_for_env(env: EnvironmentProfile) -> TrainingConfig:
$28     if env.env_id == "ALE/Freeway-v5":
$29         return TrainingConfig(total_env_steps=1_000_000)
$30     return TrainingConfig()
$31
$32
$33 def default_dqn_for_variant(variant: VariantSpec, env: EnvironmentProfile | None = None) -> DQNHypersparameters:
$34     dqn = DQNHypersparameters()
$35     if env is not None and env.env_id == "ALE/Freeway-v5":
$36         dqn = replace(
$37             dqn,
$38             batch_size=32,
$39             replay_buffer_size=25_000,
$40             learning_starts=20_000,
$41             train_frequency_env_steps=4,
$42             target_update_frequency_env_steps=8_000,
$43         )
$44     if not variant.use_replay_buffer:
$45         dqn = replace(
$46             dqn,
$47             batch_size=1,
$48             replay_buffer_size=0,
$49             learning_starts=0,
$50             train_frequency_env_steps=1,
$51             gradient_steps_per_train=1,
$52         )
$53     return dqn
$54
$55
$56 def default_exploration_for_env(env: EnvironmentProfile) -> ExplorationConfig:
$57     if env.env_id == "ALE/Freeway-v5":
$58         return ExplorationConfig(epsilon_final=0.01, epsilon_decay_env_steps=250_000)
$59     return ExplorationConfig()
$60
$61
$62 def default_optimizer_for_env(env: EnvironmentProfile) -> OptimizerConfig:
$63     if env.env_id == "ALE/Freeway-v5":
$64         return OptimizerConfig(learning_rate=6.25e-5, epsilon=1.5e-4)
$65     return OptimizerConfig()
$66
$67
$68 def default_network_for_env(env: EnvironmentProfile, variant: VariantSpec) -> NetworkConfig:
$69     if env.network_architecture == "cnn":
$70         return NetworkConfig(
$71             architecture="cnn",
$72             hidden_layers=[],
$73             dueling_aggregation="mean_subtraction" if variant.use_dueling_network else None,
$74         )
$75     return NetworkConfig(
$76         architecture="mlp",
$77         hidden_layers=[128, 128],
$78         dueling_aggregation="mean_subtraction" if variant.use_dueling_network else None,
$79     )

```



```

880
881
882 def build_run_config(
883     *,
884     env_id: str,
885     variant_name: str,
886     seed: int,
887     output_root: str | Path = "output",
888     run_id: str | None = None,
889     training_overrides: dict[str, Any] | None = None,
890     dqn_overrides: dict[str, Any] | None = None,
891     exploration_overrides: dict[str, Any] | None = None,
892     network_overrides: dict[str, Any] | None = None,
893     optimizer_overrides: dict[str, Any] | None = None,
894 ) -> RunConfig:
895     env = get_env_profile(env_id)
896     variant = get_variant_spec(variant_name)
897     output_root_path = Path(output_root)
898     resolved_run_id = run_id or f"{env.slug}_{variant.name}_seed{seed}_{_utc_timestamp()}"
899     run_dir = output_root_path / env.slug / variant.name / f"seed_{seed}" / resolved_run_id
900
901     training = default_training_for_env(env)
902     dqn = default_dqn_for_variant(variant, env)
903     exploration = default_exploration_for_env(env)
904     network = default_network_for_env(env, variant)
905     optimizer = default_optimizer_for_env(env)
906
907     if training_overrides:
908         training = replace(training, **training_overrides)
909     if dqn_overrides:
910         dqn = replace(dqn, **dqn_overrides)
911     if exploration_overrides:
912         exploration = replace(exploration, **exploration_overrides)
913     if env.env_id != "ALE/Freeway-v5" and exploration.freeway_up_bias != 0.0:
914         raise ValueError("freeway_up_bias can only be used with ALE/Freeway-v5.")
915     if network_overrides:
916         network = replace(network, **network_overrides)
917     if optimizer_overrides:
918         optimizer = replace(optimizer, **optimizer_overrides)
919
920     now = datetime.now(timezone.utc).isoformat()
921     return RunConfig(
922         run_id=resolved_run_id,
923         experiment_group="dqn_ablation_lunarlander_freeway",
924         env=env,
925         variant=variant,
926         seed=seed,
927         output_root=output_root_path,
928         run_dir=run_dir,
929         datetime_start=now,
930         datetime_end=None,
931         status="created",
932         training=training,
933         optimizer=optimizer,
934         dqn=dqn,
935         exploration=exploration,
936         network=network,
937     )
938
939
940
941 =====
942 FILE: demo.py
943 =====
944 """Run recorded demos from trained DQN checkpoints."""
945
946 from __future__ import annotations
947
948 import argparse
949 from dataclasses import dataclass
950 from pathlib import Path
951 from typing import Any
952
953 import numpy as np

```

```

954 import torch
955 from gymnasium.wrappers import RecordVideo
956 from torch import nn
957
958 from rl_analysis.config import NetworkConfig, get_variant_spec
959 from rl_analysis.envs import make_env
960 from rl_analysis.networks import build_q_network
961 from rl_analysis.utils import resolve_device
962
963
964 REQUIRED_CHECKPOINT_KEYS = (
965     "config",
966     "online_network_state_dict",
967     "run_id",
968     "variant",
969     "seed",
970 )
971
972
973 @dataclass(frozen=True)
974 class DemoResult:
975     checkpoint_path: Path
976     env_id: str
977     run_id: str
978     variant: str
979     seed: int
980     global_env_step: int | None
981     episode_returns: list[float]
982     episode_lengths: list[int]
983     video_paths: list[Path]
984
985
986 def _positive_int(value: str) -> int:
987     parsed = int(value)
988     if parsed < 1:
989         raise argparse.ArgumentTypeError("must be at least 1")
990     return parsed
991
992
993 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
994     parser = argparse.ArgumentParser(description="Run a recorded policy demo from a DQN checkpoint.")
995     parser.add_argument("--checkpoint", required=True, type=Path, help="Path to a model_step_*.pt checkpoint.")
996     parser.add_argument("--episodes", type=_positive_int, default=1, help="Number of episodes to record.")
997     parser.add_argument("--video-dir", type=Path, default=Path("output/demos"), help="Directory for MP4 output.")
998     parser.add_argument("--device", default="auto", help="Torch device: auto, cpu, cuda, cuda:N, or mps.")
999     return parser.parse_args(argv)
1000
1001
1002 def validate_checkpoint_payload(payload: Any) -> dict[str, Any]:
1003     if not isinstance(payload, dict):
1004         raise ValueError("Checkpoint payload must be a dictionary.")
1005
1006     missing = [key for key in REQUIRED_CHECKPOINT_KEYS if key not in payload]
1007     if missing:
1008         raise ValueError(f"Checkpoint payload is missing required key(s): {'', '.join(missing)}")
1009
1010     config = payload["config"]
1011     if not isinstance(config, dict):
1012         raise ValueError("Checkpoint config must be a dictionary.")
1013
1014     env_config = config.get("env")
1015     if not isinstance(env_config, dict) or not env_config.get("env_id"):
1016         raise ValueError("Checkpoint config must include config.env.env_id.")
1017
1018     network_config = config.get("network")
1019     if not isinstance(network_config, dict):
1020         raise ValueError("Checkpoint config must include config.network.")
1021
1022     if not isinstance(payload["online_network_state_dict"], dict):
1023         raise ValueError("Checkpoint online_network_state_dict must be a state-dict dictionary.")
1024
1025     get_variant_spec(str(payload["variant"]))
1026     return payload
1027

```

```

1028
1029 def load_checkpoint_payload(checkpoint_path: Path) -> dict[str, Any]:
1030     resolved_path = checkpoint_path.expanduser()
1031     if not resolved_path.exists():
1032         raise FileNotFoundError(f"Checkpoint does not exist: {resolved_path}")
1033     payload = torch.load(resolved_path, map_location="cpu", weights_only=False)
1034     return validate_checkpoint_payload(payload)
1035
1036
1037 def network_config_from_checkpoint(payload: dict[str, Any]) -> NetworkConfig:
1038     validate_checkpoint_payload(payload)
1039     allowed_fields = set(NetworkConfig.__dataclass_fields__)
1040     raw_config = payload["config"]["network"]
1041     return NetworkConfig(**{key: value for key, value in raw_config.items() if key in allowed_fields})
1042
1043
1044 def env_id_from_checkpoint(payload: dict[str, Any]) -> str:
1045     validate_checkpoint_payload(payload)
1046     return str(payload["config"]["env"]["env_id"])
1047
1048
1049 def build_network_from_checkpoint(
1050     payload: dict[str, Any],
1051     observation_shape: tuple[int, ...],
1052     action_dim: int,
1053     device: torch.device,
1054 ) -> nn.Module:
1055     network = build_q_network(observation_shape, action_dim, network_config_from_checkpoint(payload))
1056     network.load_state_dict(payload["online_network_state_dict"])
1057     network.to(device)
1058     network.eval()
1059     return network
1060
1061
1062 def select_greedy_action(network: nn.Module, observation: np.ndarray, device: torch.device) -> int:
1063     obs_tensor = torch.as_tensor(np.asarray(observation), dtype=torch.float32, device=device).unsqueeze(0)
1064     with torch.no_grad():
1065         q_values = network(obs_tensor)
1066     return int(torch.argmax(q_values, dim=1).item())
1067
1068
1069 def run_demo_episode(env, network: nn.Module, device: torch.device, *, seed: int) -> tuple[float, int]:
1070     observation, _info = env.reset(seed=seed)
1071     done = False
1072     episode_return = 0.0
1073     episode_length = 0
1074     while not done:
1075         action = select_greedy_action(network, observation, device)
1076         observation, reward, terminated, truncated, _info = env.step(action)
1077         episode_return += float(reward)
1078         episode_length += 1
1079         done = bool(terminated or truncated)
1080     return episode_return, episode_length
1081
1082
1083 def _safe_name(value: str) -> str:
1084     safe = "".join(char if char.isalnum() or char in {"-", "_"} else "_" for char in value)
1085     return safe[:80] or "checkpoint"
1086
1087
1088 def run_demo(
1089     *,
1090     checkpoint_path: Path,
1091     episodes: int,
1092     video_dir: Path,
1093     device_name: str,
1094 ) -> DemoResult:
1095     payload = load_checkpoint_payload(checkpoint_path)
1096     env_id = env_id_from_checkpoint(payload)
1097     run_id = str(payload["run_id"])
1098     variant = str(payload["variant"])
1099     seed = int(payload["seed"])
1100     global_env_step = payload.get("global_env_step")
1101     if global_env_step is not None:

```

```

1/02     global_env_step = int(global_env_step)
1/03
1/04     device = resolve_device(device_name)
1/05     video_dir = video_dir.expanduser()
1/06     video_dir.mkdir(parents=True, exist_ok=True)
1/07     existing_videos = set(video_dir.glob("*.mp4"))
1/08
1/09     env = None
1/10     episode_returns: list[float] = []
1/11     episode_lengths: list[int] = []
1/12     try:
1/13         env = make_env(env_id, seed=seed, render_mode="rgb_array")
1/14         env = RecordVideo(
1/15             env,
1/16             video_folder=str(video_dir),
1/17             episode_trigger=lambda _episode_id: True,
1/18             name_prefix=f"{_safe_name(run_id)}-demo",
1/19             disable_logger=True,
1/20         )
1/21         observation_shape = tuple(int(dim) for dim in env.observation_space.shape)
1/22         action_dim = int(env.action_space.n)
1/23         network = build_network_from_checkpoint(payload, observation_shape, action_dim, device)
1/24
1/25         for episode_index in range(episodes):
1/26             episode_return, episode_length = run_demo_episode(
1/27                 env,
1/28                 network,
1/29                 device,
1/30                 seed=seed + 200_000 + episode_index,
1/31             )
1/32             episode_returns.append(episode_return)
1/33             episode_lengths.append(episode_length)
1/34     finally:
1/35         if env is not None:
1/36             env.close()
1/37
1/38     video_paths = sorted(path for path in video_dir.glob("*.mp4") if path not in existing_videos)
1/39     return DemoResult(
1/40         checkpoint_path=checkpoint_path.expanduser(),
1/41         env_id=env_id,
1/42         run_id=run_id,
1/43         variant=variant,
1/44         seed=seed,
1/45         global_env_step=global_env_step,
1/46         episode_returns=episode_returns,
1/47         episode_lengths=episode_lengths,
1/48         video_paths=video_paths,
1/49     )
1/50
1/51
1/52 def print_summary(result: DemoResult) -> None:
1/53     print(f"Checkpoint: {result.checkpoint_path}")
1/54     print(f"Run: {result.run_id} ({result.env_id}, {result.variant}, seed={result.seed})")
1/55     if result.global_env_step is not None:
1/56         print(f"Checkpoint step: {result.global_env_step}")
1/57     print(f"Episode returns: {result.episode_returns}")
1/58     print(f"Episode lengths: {result.episode_lengths}")
1/59     if result.video_paths:
1/60         print("Videos:")
1/61         for path in result.video_paths:
1/62             print(f"  {path}")
1/63     else:
1/64         print("Videos: none found")
1/65
1/66
1/67 def main(argv: list[str] | None = None) -> None:
1/68     args = parse_args(argv)
1/69     result = run_demo(
1/70         checkpoint_path=args.checkpoint,
1/71         episodes=args.episodes,
1/72         video_dir=args.video_dir,
1/73         device_name=args.device,
1/74     )
1/75     print_summary(result)

```

```

1176
1177
1178 if __name__ == "__main__":
1179     main()
1180
1181
1182
1183 =====
1184 FILE: envs.py
1185 =====
1186 """Environment creation and environment-specific metrics."""
1187
1188 from __future__ import annotations
1189
1190 from dataclasses import asdict
1191 from typing import Any
1192
1193 import gymnasium as gym
1194 import numpy as np
1195
1196 from rl_analysis.config import EnvironmentProfile, get_env_profile
1197
1198
1199 FREEWAY_ACTION_NOOP = 0
1200 FREEWAY_ACTION_UP = 1
1201 FREEWAY_ACTION_DOWN = 2
1202
1203
1204 def make_env(env_id: str | EnvironmentProfile, *, seed: int | None = None, render_mode: str | None = None):
1205     profile = get_env_profile(env_id) if isinstance(env_id, str) else env_id
1206     if profile.env_id == "ALE/Freeway-v5":
1207         env = _make_freeway_env(profile, render_mode=render_mode)
1208         if seed is not None:
1209             env.reset(seed=seed)
1210             env.action_space.seed(seed)
1211             env.observation_space.seed(seed)
1212         return env
1213     kwargs: dict[str, Any] = {}
1214     if render_mode is not None:
1215         kwargs["render_mode"] = render_mode
1216     env = gym.make(profile.env_id, **kwargs)
1217     if seed is not None:
1218         env.reset(seed=seed)
1219         env.action_space.seed(seed)
1220         env.observation_space.seed(seed)
1221     return env
1222
1223
1224 def _register_ale() -> None:
1225     try:
1226         import ale_py
1227     except ImportError as exc:
1228         raise gym.error.DependencyNotInstalled(
1229             'Atari Freeway requires ale-py. Install with 'uv pip install "gymnasium[atari]" ale-py'.'
1230         ) from exc
1231
1232     register_envs = getattr(gym, "register_envs", None)
1233     if register_envs is not None:
1234         register_envs(ale_py)
1235
1236
1237 def _make_freeway_env(profile: EnvironmentProfile, *, render_mode: str | None = None):
1238     from gymnasium.wrappers import AtariPreprocessing, FrameStackObservation, TransformReward
1239
1240     _register_ale()
1241     kwargs: dict[str, Any] = {
1242         "frameskip": profile.raw_frame_skip or 1,
1243         "repeat_action_probability": profile.repeat_action_probability
1244         if profile.repeat_action_probability is not None
1245         else 0.25,
1246         "full_action_space": bool(profile.full_action_space),
1247     }
1248     if render_mode is not None:
1249         kwargs["render_mode"] = render_mode

```

```

1250     env = gym.make(profile.env_id, **kwargs)
1251     env = AtariPreprocessing(
1252         env,
1253         noop_max=profile.noop_max or 0,
1254         frame_skip=profile.frame_skip or 1,
1255         screen_size=profile.resize_shape or 84,
1256         terminal_on_life_loss=bool(profile.terminal_on_life_loss),
1257         grayscale_obs=bool(profile.grayscale),
1258         grayscale_newaxis=False,
1259         scale_obs=False,
1260     )
1261     if profile.reward_clipping:
1262         env = TransformReward(env, lambda reward: float(np.sign(float(reward))))
1263     if profile.frame_stack and profile.frame_stack > 1:
1264         env = FrameStackObservation(env, stack_size=profile.frame_stack)
1265     return env
1266
1267
1268 def environment_config_dict(profile: EnvironmentProfile) -> dict[str, Any]:
1269     payload = asdict(profile)
1270     payload["resize_shape"] = list(profile.resize_shape) if profile.resize_shape is not None else None
1271     payload["score_thresholds"] = list(profile.score_thresholds)
1272     return payload
1273
1274
1275 def freeway_up_biased_action_probabilities(action_dim: int, freeway_up_bias: float) -> np.ndarray:
1276     if action_dim <= FREEWAY_ACTION_UP:
1277         raise ValueError("Freeway up-biased exploration requires an action space with an up action.")
1278     if not 0.0 <= freeway_up_bias <= 1.0:
1279         raise ValueError("freeway_up_bias must be between 0.0 and 1.0.")
1280
1281     base_probability = (1.0 - freeway_up_bias) / action_dim
1282     probabilities = np.full(action_dim, base_probability, dtype=np.float64)
1283     probabilities[FREEWAY_ACTION_UP] += freeway_up_bias
1284     return probabilities
1285
1286
1287 def lunarlander_episode_metrics(
1288     *,
1289     episode_return: float,
1290     episode_length: int,
1291     action_counts: np.ndarray,
1292     final_observation: np.ndarray,
1293     terminated: bool,
1294     truncated: bool,
1295 ) -> dict[str, Any]:
1296     safe_length = max(1, episode_length)
1297     final = np.asarray(final_observation, dtype=np.float32)
1298     both_legs = bool(final[6] > 0.5 and final[7] > 0.5) if final.shape[0] >= 8 else False
1299     one_leg = bool((final[6] > 0.5) ^ (final[7] > 0.5)) if final.shape[0] >= 8 else False
1300     return {
1301         "landing_success": bool(episode_return >= 200.0),
1302         "crash": bool(terminated and episode_return < 0.0),
1303         "timeout": bool(truncated),
1304         "fuel_proxy_main_engine_count": int(action_counts[2]) if action_counts.size > 2 else 0,
1305         "fuel_proxy_side_engine_count": int(action_counts[1] + action_counts[3]) if action_counts.size > 3 else 0,
1306         "main_engine_action_fraction": float(action_counts[2] / safe_length) if action_counts.size > 2 else 0.0,
1307         "side_engine_action_fraction": float((action_counts[1] + action_counts[3]) / safe_length)
1308         if action_counts.size > 3
1309         else 0.0,
1310         "final_x_position": float(final[0]) if final.shape[0] > 0 else None,
1311         "final_y_position": float(final[1]) if final.shape[0] > 1 else None,
1312         "final_x_velocity": float(final[2]) if final.shape[0] > 2 else None,
1313         "final_y_velocity": float(final[3]) if final.shape[0] > 3 else None,
1314         "final_angle": float(final[4]) if final.shape[0] > 4 else None,
1315         "final_angular_velocity": float(final[5]) if final.shape[0] > 5 else None,
1316         "final_left_leg_contact": bool(final[6] > 0.5) if final.shape[0] > 6 else None,
1317         "final_right_leg_contact": bool(final[7] > 0.5) if final.shape[0] > 7 else None,
1318         "both_legs_contact": both_legs,
1319         "one_leg_contact": one_leg,
1320     }
1321
1322
1323

```

```

1324 def freeway_episode_metrics(
1325     *,
1326     episode_return: float,
1327     episode_length: int,
1328     action_counts: np.ndarray,
1329     max_score_so_far: float,
1330 ) -> dict[str, Any]:
1331     safe_length = max(1, episode_length)
1332     score = float(episode_return)
1333     return {
1334         "score": int(round(score)),
1335         "score_raw": score,
1336         "zero_score_episode": bool(score <= 0.0),
1337         "max_score_so_far": float(max_score_so_far),
1338         "action_noop_fraction": float(action_counts[FREEWAY_ACTION_NOOP] / safe_length)
1339         if action_counts.size > FREEWAY_ACTION_NOOP
1340         else 0.0,
1341         "action_up_fraction": float(action_counts[FREEWAY_ACTION_UP] / safe_length)
1342         if action_counts.size > FREEWAY_ACTION_UP
1343         else 0.0,
1344         "action_down_fraction": float(action_counts[FREEWAY_ACTION_DOWN] / safe_length)
1345         if action_counts.size > FREEWAY_ACTION_DOWN
1346         else 0.0,
1347     }
1348
1349
1350
1351 =====
1352 FILE: logging_utils.py
1353 =====
1354 """JSON and JSONL logging helpers for experiment artifacts."""
1355
1356 from __future__ import annotations
1357
1358 import time
1359 from pathlib import Path
1360 from typing import Any
1361
1362 import torch
1363
1364 from rl_analysis.utils import to_jsonable, write_json
1365
1366
1367 class JSONLLogger:
1368     def __init__(self, path: Path):
1369         self.path = path
1370         self.path.parent.mkdir(parents=True, exist_ok=True)
1371         self._handle = self.path.open("a", encoding="utf-8")
1372
1373     def write(self, row: dict[str, Any]) -> None:
1374         import json
1375
1376         self._handle.write(json.dumps(to_jsonable(row), sort_keys=True) + "\n")
1377         self._handle.flush()
1378
1379     def close(self) -> None:
1380         self._handle.close()
1381
1382     def __enter__(self) -> "JSONLLogger":
1383         return self
1384
1385     def __exit__(self, exc_type: object, exc: object, tb: object) -> None:
1386         self.close()
1387
1388
1389 class RunLoggers:
1390     def __init__(self, run_dir: Path):
1391         self.run_dir = run_dir
1392         self.train_episode = JSONLLogger(run_dir / "train_episode_metrics.jsonl")
1393         self.train_update = JSONLLogger(run_dir / "train_update_metrics.jsonl")
1394         self.eval = JSONLLogger(run_dir / "eval_metrics.jsonl")
1395         self.checkpoint = JSONLLogger(run_dir / "checkpoint_metrics.jsonl")
1396         self.system = JSONLLogger(run_dir / "system_metrics.jsonl")
1397

```

```

1398     def close(self) -> None:
1399         self.train_episode.close()
1400         self.train_update.close()
1401         self.eval.close()
1402         self.checkpoint.close()
1403         self.system.close()
1404
1405     def __enter__(self) -> "RunLoggers":
1406         return self
1407
1408     def __exit__(self, exc_type: object, exc: object, tb: object) -> None:
1409         self.close()
1410
1411
1412 def write_run_config(run_dir: Path, payload: dict[str, Any]) -> None:
1413     write_json(run_dir / "run_config.json", payload)
1414
1415
1416 def _mps_memory_gb(name: str) -> float | None:
1417     if not hasattr(torch, "mps"):
1418         return None
1419     reader = getattr(torch.mps, name, None)
1420     if reader is None:
1421         return None
1422     try:
1423         return float(reader() / 1024**3)
1424     except Exception:
1425         return None
1426
1427
1428 def collect_system_metrics(
1429     *,
1430     run_id: str,
1431     global_env_step: int,
1432     device: torch.device,
1433     wall_time_start: float,
1434     env_steps_per_second: float | None,
1435     updates_per_second: float | None,
1436     replay_buffer_memory_gb: float | None,
1437 ) -> dict[str, Any]:
1438     cpu_percent = None
1439     ram_used_gb = None
1440     try:
1441         import psutil
1442
1443         cpu_percent = float(psutil.cpu_percent(interval=None))
1444         ram_used_gb = float(psutil.virtual_memory().used / 1024**3)
1445     except Exception:
1446         pass
1447
1448     gpu_util_percent = None
1449     gpu_memory_used_gb = None
1450     mps_memory_allocated_gb = None
1451     mps_driver_allocated_gb = None
1452     mps_recommended_max_memory_gb = None
1453     if device.type == "cuda" and torch.cuda.is_available():
1454         gpu_memory_used_gb = float(torch.cuda.memory_allocated() / 1024**3)
1455     elif device.type == "mps":
1456         mps_memory_allocated_gb = _mps_memory_gb("current_allocated_memory")
1457         mps_driver_allocated_gb = _mps_memory_gb("driver_allocated_memory")
1458         mps_recommended_max_memory_gb = _mps_memory_gb("recommended_max_memory")
1459         gpu_memory_used_gb = mps_memory_allocated_gb
1460
1461     return {
1462         "run_id": run_id,
1463         "global_env_step": global_env_step,
1464         "device": str(device),
1465         "wall_time_elapsed_sec": time.perf_counter() - wall_time_start,
1466         "env_steps_per_second": env_steps_per_second,
1467         "updates_per_second": updates_per_second,
1468         "cpu_percent": cpu_percent,
1469         "ram_used_gb": ram_used_gb,
1470         "gpu_util_percent": gpu_util_percent,
1471         "gpu_memory_used_gb": gpu_memory_used_gb,

```



```

1472     "mps_memory_allocated_gb": mps_memory_allocated_gb,
1473     "mps_driver_allocated_gb": mps_driver_allocated_gb,
1474     "mps_recommended_max_memory_gb": mps_recommended_max_memory_gb,
1475     "replay_buffer_memory_gb": replay_buffer_memory_gb,
1476 }
1477
1478
1479
1480 =====
1481 FILE: networks.py
1482 =====
1483 """Q-network definitions."""
1484
1485 from __future__ import annotations
1486
1487 from typing import Any
1488
1489 import torch
1490 from torch import nn
1491
1492 from rl_analysis.config import NetworkConfig
1493
1494
1495 def _activation(name: str) -> type[nn.Module]:
1496     key = name.lower()
1497     if key == "relu":
1498         return nn.ReLU
1499     if key == "tanh":
1500         return nn.Tanh
1501     if key == "gelu":
1502         return nn.GELU
1503     raise ValueError(f"Unsupported activation {name!r}.")
1504
1505
1506 def _orthogonal_init(module: nn.Module) -> None:
1507     if isinstance(module, nn.Linear | nn.Conv2d):
1508         nn.init.orthogonal_(module.weight, gain=nn.init.calculate_gain("relu"))
1509         if module.bias is not None:
1510             nn.init.zeros_(module.bias)
1511
1512
1513 class QNetwork(nn.Module):
1514     def __init__(
1515         self,
1516         observation_dim: int,
1517         action_dim: int,
1518         hidden_layers: list[int],
1519         activation: str = "relu",
1520         dueling: bool = False,
1521     ):
1522         super().__init__()
1523         self.observation_dim = int(observation_dim)
1524         self.action_dim = int(action_dim)
1525         self.dueling = bool(dueling)
1526
1527         act = _activation(activation)
1528         layers: list[nn.Module] = []
1529         in_dim = self.observation_dim
1530         for hidden_dim in hidden_layers:
1531             layers.append(nn.Linear(in_dim, hidden_dim))
1532             layers.append(act())
1533             in_dim = hidden_dim
1534         self.feature_extractor = nn.Sequential(*layers)
1535
1536         if self.dueling:
1537             self.value_head = nn.Linear(in_dim, 1)
1538             self.advantage_head = nn.Linear(in_dim, self.action_dim)
1539         else:
1540             self.q_head = nn.Linear(in_dim, self.action_dim)
1541
1542         self.apply(_orthogonal_init)
1543
1544     def forward(self, observations: torch.Tensor, return_streams: bool = False):
1545         if observations.ndim == 1:

```

```

1546         observations = observations.unsqueeze(0)
1547         features = self.feature_extractor(observations.float())
1548         if self.dueling:
1549             value = self.value_head(features)
1550             advantage = self.advantage_head(features)
1551             q_values = value + advantage - advantage.mean(dim=1, keepdim=True)
1552             if return_streams:
1553                 return q_values, {"value": value, "advantage": advantage}
1554             return q_values
1555         q_values = self.q_head(features)
1556         if return_streams:
1557             return q_values, {"value": None, "advantage": None}
1558         return q_values
1559
1560
1561 class CnnQNetwork(nn.Module):
1562     def __init__(
1563         self,
1564         observation_shape: tuple[int, ...],
1565         action_dim: int,
1566         conv_layers: list[dict[str, int]],
1567         fully_connected_layers: list[int],
1568         activation: str = "relu",
1569         dueling: bool = False,
1570     ):
1571         super().__init__()
1572         if len(observation_shape) != 3:
1573             raise ValueError(f"CNN QNetwork expects stacked image observations, got shape {observation_shape}.")
1574         self.observation_shape = tuple(int(x) for x in observation_shape)
1575         self.action_dim = int(action_dim)
1576         self.dueling = bool(dueling)
1577
1578         act = _activation(activation)
1579         channels = self.observation_shape[0]
1580         conv_modules: list[nn.Module] = []
1581         for layer in conv_layers:
1582             conv_modules.append(
1583                 nn.Conv2d(
1584                     channels,
1585                     int(layer["out_channels"]),
1586                     kernel_size=int(layer["kernel_size"]),
1587                     stride=int(layer["stride"]),
1588                 )
1589             )
1590             conv_modules.append(act())
1591             channels = int(layer["out_channels"])
1592         self.conv = nn.Sequential(*conv_modules)
1593
1594         with torch.no_grad():
1595             dummy = torch.zeros(1, *self.observation_shape)
1596             conv_output_dim = int(self.conv(dummy).flatten(start_dim=1).shape[1])
1597
1598         fc_modules: list[nn.Module] = []
1599         in_dim = conv_output_dim
1600         for hidden_dim in fully_connected_layers:
1601             fc_modules.append(nn.Linear(in_dim, hidden_dim))
1602             fc_modules.append(act())
1603             in_dim = hidden_dim
1604         self.feature_head = nn.Sequential(*fc_modules)
1605
1606         if self.dueling:
1607             self.value_head = nn.Linear(in_dim, 1)
1608             self.advantage_head = nn.Linear(in_dim, self.action_dim)
1609         else:
1610             self.q_head = nn.Linear(in_dim, self.action_dim)
1611
1612         self.apply(_orthogonal_init)
1613
1614     def forward(self, observations: torch.Tensor, return_streams: bool = False):
1615         if observations.ndim == 3:
1616             observations = observations.unsqueeze(0)
1617         if observations.ndim != 4:
1618             raise ValueError(f"CNN QNetwork expects observations with 3 or 4 dims, got {observations.ndim}.")
1619         features = self.conv(observations.float() / 255.0).flatten(start_dim=1)

```

```

1620     features = self.feature_head(features)
1621     if self.dueling:
1622         value = self.value_head(features)
1623         advantage = self.advantage_head(features)
1624         q_values = value + advantage - advantage.mean(dim=1, keepdim=True)
1625         if return_streams:
1626             return q_values, {"value": value, "advantage": advantage}
1627         return q_values
1628     q_values = self.q_head(features)
1629     if return_streams:
1630         return q_values, {"value": None, "advantage": None}
1631     return q_values
1632
1633
1634 def build_q_network(observation_shape: tuple[int, ...], action_dim: int, config: NetworkConfig) -> nn.Module:
1635     if config.architecture == "cnn":
1636         return CnnQNetwork(
1637             observation_shape=observation_shape,
1638             action_dim=action_dim,
1639             conv_layers=config.conv_layers,
1640             fully_connected_layers=config.fully_connected_layers,
1641             activation=config.activation,
1642             dueling=config.dueling_aggregation is not None,
1643         )
1644     if config.architecture != "mlp":
1645         raise ValueError(f"Unsupported network architecture {config.architecture!r}.")
1646     if len(observation_shape) != 1:
1647         raise ValueError(f"MLP QNetwork expects a flat vector observation, got shape {observation_shape}.")
1648     return QNetwork(
1649         observation_dim=observation_shape[0],
1650         action_dim=action_dim,
1651         hidden_layers=config.hidden_layers,
1652         activation=config.activation,
1653         dueling=config.dueling_aggregation is not None,
1654     )
1655
1656
1657 def network_architecture_dict(config: NetworkConfig, num_parameters: int) -> dict[str, Any]:
1658     if config.architecture == "cnn":
1659         return {
1660             "architecture": config.architecture,
1661             "conv_layers": config.conv_layers,
1662             "fully_connected_layers": config.fully_connected_layers,
1663             "activation": config.activation,
1664             "dueling_aggregation": config.dueling_aggregation,
1665             "num_parameters": num_parameters,
1666         }
1667     return {
1668         "architecture": config.architecture,
1669         "hidden_layers": config.hidden_layers,
1670         "activation": config.activation,
1671         "dueling_aggregation": config.dueling_aggregation,
1672         "num_parameters": num_parameters,
1673         "initializer": config.initializer,
1674     }
1675
1676
1677
1678 =====
1679 FILE: replay.py
1680 =====
1681 """Replay-buffer data structures."""
1682
1683 from __future__ import annotations
1684
1685 from dataclasses import dataclass
1686 from typing import Any
1687
1688 import numpy as np
1689
1690
1691 @dataclass
1692 class Transition:
1693     state: np.ndarray

```

```

1094     action: int
1095     reward: float
1096     next_state: np.ndarray
1097     terminated: bool
1098     truncated: bool
1099     global_step: int
1100     episode_index: int
1101
1102
1103 @dataclass
1104 class TransitionBatch:
1105     states: np.ndarray
1106     actions: np.ndarray
1107     rewards: np.ndarray
1108     next_states: np.ndarray
1109     terminated: np.ndarray
1110     truncated: np.ndarray
1111     steps: np.ndarray
1112     episode_indices: np.ndarray
1113
1114     @property
1115     def done_for_logging(self) -> np.ndarray:
1116         return np.logical_or(self.terminated, self.truncated)
1117
1118
1119 def transition_to_batch(transition: Transition) -> TransitionBatch:
1120     return TransitionBatch(
1121         states=np.expand_dims(np.asarray(transition.state, dtype=np.float32), axis=0),
1122         actions=np.asarray([transition.action], dtype=np.int64),
1123         rewards=np.asarray([transition.reward], dtype=np.float32),
1124         next_states=np.expand_dims(np.asarray(transition.next_state, dtype=np.float32), axis=0),
1125         terminated=np.asarray([transition.terminated], dtype=np.bool_),
1126         truncated=np.asarray([transition.truncated], dtype=np.bool_),
1127         steps=np.asarray([transition.global_step], dtype=np.int64),
1128         episode_indices=np.asarray([transition.episode_index], dtype=np.int64),
1129     )
1130
1131
1132 class ReplayBuffer:
1133     def __init__(
1134         self,
1135         capacity: int,
1136         observation_shape: tuple[int, ...],
1137         observation_dtype: np.dtype | type | str = np.float32,
1138     ):
1139         if capacity <= 0:
1140             raise ValueError("ReplayBuffer capacity must be positive.")
1141         self.capacity = int(capacity)
1142         self.observation_shape = tuple(observation_shape)
1143         self.observation_dtype = np.dtype(observation_dtype)
1144         self.states = np.zeros((self.capacity, *self.observation_shape), dtype=self.observation_dtype)
1145         self.next_states = np.zeros((self.capacity, *self.observation_shape), dtype=self.observation_dtype)
1146         self.actions = np.zeros(self.capacity, dtype=np.int64)
1147         self.rewards = np.zeros(self.capacity, dtype=np.float32)
1148         self.terminated = np.zeros(self.capacity, dtype=np.bool_)
1149         self.truncated = np.zeros(self.capacity, dtype=np.bool_)
1150         self.steps = np.zeros(self.capacity, dtype=np.int64)
1151         self.episode_indices = np.zeros(self.capacity, dtype=np.int64)
1152         self._pos = 0
1153         self._size = 0
1154
1155     def __len__(self) -> int:
1156         return self._size
1157
1158     def add(self, transition: Transition) -> None:
1159         idx = self._pos
1160         self.states[idx] = np.asarray(transition.state, dtype=self.observation_dtype)
1161         self.next_states[idx] = np.asarray(transition.next_state, dtype=self.observation_dtype)
1162         self.actions[idx] = transition.action
1163         self.rewards[idx] = transition.reward
1164         self.terminated[idx] = transition.terminated
1165         self.truncated[idx] = transition.truncated
1166         self.steps[idx] = transition.global_step
1167         self.episode_indices[idx] = transition.episode_index

```

```

1768         self._pos = (self._pos + 1) % self.capacity
1769         self._size = min(self._size + 1, self.capacity)
1770
1771     def sample(self, batch_size: int, rng: np.random.Generator) -> TransitionBatch:
1772         if self._size == 0:
1773             raise ValueError("Cannot sample from an empty replay buffer.")
1774         if batch_size > self._size:
1775             raise ValueError(f"Requested batch_size={batch_size}, but buffer only has {self._size} transitions.")
1776         indices = rng.choice(self._size, size=batch_size, replace=False)
1777         return TransitionBatch(
1778             states=self.states[indices].copy(),
1779             actions=self.actions[indices].copy(),
1780             rewards=self.rewards[indices].copy(),
1781             next_states=self.next_states[indices].copy(),
1782             terminated=self.terminated[indices].copy(),
1783             truncated=self.truncated[indices].copy(),
1784             steps=self.steps[indices].copy(),
1785             episode_indices=self.episode_indices[indices].copy(),
1786         )
1787
1788     def memory_gb(self) -> float:
1789         total_bytes = (
1790             self.states.nbytes
1791             + self.next_states.nbytes
1792             + self.actions.nbytes
1793             + self.rewards.nbytes
1794             + self.terminated.nbytes
1795             + self.truncated.nbytes
1796             + self.steps.nbytes
1797             + self.episode_indices.nbytes
1798         )
1799         return float(total_bytes / 1024**3)
1800
1801     def replay_diagnostics(
1802         *,
1803         batch: TransitionBatch | None,
1804         replay_buffer: ReplayBuffer | None,
1805         current_step: int,
1806         replay_enabled: bool,
1807     ) -> dict[str, Any]:
1808         if not replay_enabled:
1809             return {
1810                 "enabled": False,
1811                 "buffer_size": None,
1812                 "buffer_capacity": 0,
1813                 "buffer_fill_fraction": None,
1814                 "sample_age_mean": 0.0,
1815                 "sample_age_std": 0.0,
1816                 "sample_age_min": 0,
1817                 "sample_age_max": 0,
1818                 "sample_age_p50": 0.0,
1819                 "sample_age_p90": 0.0,
1820                 "sample_reward_mean": float(np.mean(batch.rewards)) if batch is not None else None,
1821                 "sample_reward_std": float(np.std(batch.rewards)) if batch is not None else None,
1822                 "sample_done_fraction": float(np.mean(batch.done_for_logging)) if batch is not None else None,
1823                 "sample_unique_episode_count": 1 if batch is not None else None,
1824                 "sample_consecutive_transition_fraction": 1.0 if batch is not None else None,
1825             }
1826         size = len(replay_buffer) if replay_buffer is not None else 0
1827         capacity = replay_buffer.capacity if replay_buffer is not None else 0
1828         if batch is None:
1829             return {
1830                 "enabled": True,
1831                 "buffer_size": size,
1832                 "buffer_capacity": capacity,
1833                 "buffer_fill_fraction": float(size / capacity) if capacity else None,
1834                 "sample_age_mean": None,
1835                 "sample_age_std": None,
1836                 "sample_age_min": None,
1837                 "sample_age_max": None,
1838                 "sample_age_p50": None,
1839             }

```

```

1842         "sample_age_p90": None,
1843         "sample_reward_mean": None,
1844         "sample_reward_std": None,
1845         "sample_done_fraction": None,
1846         "sample_unique_episode_count": None,
1847         "sample_consecutive_transition_fraction": None,
1848     }
1849
1850     ages = np.maximum(0, current_step - batch.steps)
1851     sorted_steps = np.sort(batch.steps)
1852     consecutive = np.diff(sorted_steps) == 1
1853     return {
1854         "enabled": True,
1855         "buffer_size": size,
1856         "buffer_capacity": capacity,
1857         "buffer_fill_fraction": float(size / capacity) if capacity else None,
1858         "sample_age_mean": float(np.mean(ages)),
1859         "sample_age_std": float(np.std(ages)),
1860         "sample_age_min": int(np.min(ages)),
1861         "sample_age_max": int(np.max(ages)),
1862         "sample_age_p50": float(np.percentile(ages, 50)),
1863         "sample_age_p90": float(np.percentile(ages, 90)),
1864         "sample_reward_mean": float(np.mean(batch.rewards)),
1865         "sample_reward_std": float(np.std(batch.rewards)),
1866         "sample_done_fraction": float(np.mean(batch.done_for_logging)),
1867         "sample_unique_episode_count": int(np.unique(batch.episode_indices).size),
1868         "sample_consecutive_transition_fraction": float(np.mean(consecutive)) if consecutive.size else 0.0,
1869     }
1870
1871
1872
1873 =====
1874 FILE: train.py
1875 =====
1876 """Command line entrypoint for training DQN experiments."""
1877
1878 from __future__ import annotations
1879
1880 import argparse
1881 import multiprocessing as mp
1882 import sys
1883 from concurrent.futures import ProcessPoolExecutor, as_completed
1884 from dataclasses import dataclass
1885 from pathlib import Path
1886 from typing import Any
1887
1888 from rl_analysis.config import VARIANT_NAMES, RunConfig, build_run_config
1889 from rl_analysis.trainer import DQNTrainer
1890
1891
1892 DEFAULT_SEEDS = [0, 1, 2, 3, 4]
1893
1894
1895 @dataclass(frozen=True)
1896 class TrainResult:
1897     run_id: str
1898     run_dir: Path
1899     variant: str
1900     seed: int
1901     final_eval_return_mean: Any
1902
1903
1904 def _positive_int(value: str) -> int:
1905     parsed = int(value)
1906     if parsed < 1:
1907         raise argparse.ArgumentTypeError("must be at least 1")
1908     return parsed
1909
1910
1911 def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
1912     parser = argparse.ArgumentParser(description="Train DQN ablations on Gymnasium environments.")
1913     parser.add_argument("--env", default="LunarLander-v3", help="Environment id or alias.")
1914     parser.add_argument("--variant", default="dqn", choices=[*VARIANT_NAMES, "all"], help="Variant to train.")
1915     parser.add_argument("--seed", type=int, default=0, help="Seed used when --variant is not all.")

```

```

1916 parser.add_argument(
1917     "--seeds",
1918     type=int,
1919     nargs="+",
1920     default=None,
1921     help="Seeds to run. Overrides --seed; defaults to 0 1 2 3 4 for --variant all.",
1922 )
1923 parser.add_argument("--output-root", default="output", help="Root directory for experiment artifacts.")
1924 parser.add_argument("--device", default="auto", help="Torch device: auto, cpu, cuda, cuda:N, or mps.")
1925 parser.add_argument(
1926     "--max-seed-workers",
1927     type=_positive_int,
1928     default=None,
1929     help="Maximum seed processes to run concurrently for each variant. Defaults to all selected seeds.",
1930 )
1931 parser.add_argument("--serial", action="store_true", help="Run selected seeds one at a time.")
1932
1933 parser.add_argument("--total-env-steps", type=int, default=None)
1934 parser.add_argument("--eval-frequency-env-steps", type=int, default=None)
1935 parser.add_argument("--checkpoint-frequency-env-steps", type=int, default=None)
1936 parser.add_argument("--num-eval-episodes", type=int, default=None)
1937 parser.add_argument("--max-episodes", type=int, default=None)
1938 parser.add_argument("--update-log-frequency", type=int, default=None)
1939 parser.add_argument("--system-metrics-frequency-env-steps", type=int, default=None)
1940
1941 parser.add_argument("--learning-starts", type=int, default=None)
1942 parser.add_argument("--batch-size", type=int, default=None)
1943 parser.add_argument("--replay-buffer-size", type=int, default=None)
1944 parser.add_argument("--train-frequency-env-steps", type=int, default=None)
1945 parser.add_argument("--target-update-frequency-env-steps", type=int, default=None)
1946 parser.add_argument("--gamma", type=float, default=None)
1947 parser.add_argument("--learning-rate", type=float, default=None)
1948 parser.add_argument("--epsilon-decay-env-steps", type=int, default=None)
1949 parser.add_argument("--epsilon-final", type=float, default=None)
1950 parser.add_argument(
1951     "--freeway-up-bias",
1952     type=float,
1953     default=0.0,
1954     help="Bias Freeway training exploration toward the up action. Must be in [0, 1].",
1955 )
1956 return parser.parse_args(argv)
1957
1958
1959 def _training_overrides(args: argparse.Namespace) -> dict:
1960     mapping = {
1961         "total_env_steps": args.total_env_steps,
1962         "eval_frequency_env_steps": args.eval_frequency_env_steps,
1963         "checkpoint_frequency_env_steps": args.checkpoint_frequency_env_steps,
1964         "num_eval_episodes": args.num_eval_episodes,
1965         "max_episodes": args.max_episodes,
1966         "update_log_frequency": args.update_log_frequency,
1967         "system_metrics_frequency_env_steps": args.system_metrics_frequency_env_steps,
1968         "device": args.device,
1969     }
1970     return {key: value for key, value in mapping.items() if value is not None}
1971
1972
1973 def _dqn_overrides(args: argparse.Namespace) -> dict:
1974     mapping = {
1975         "learning_starts": args.learning_starts,
1976         "batch_size": args.batch_size,
1977         "replay_buffer_size": args.replay_buffer_size,
1978         "train_frequency_env_steps": args.train_frequency_env_steps,
1979         "target_update_frequency_env_steps": args.target_update_frequency_env_steps,
1980         "gamma": args.gamma,
1981     }
1982     return {key: value for key, value in mapping.items() if value is not None}
1983
1984
1985 def _optimizer_overrides(args: argparse.Namespace) -> dict:
1986     mapping = {"learning_rate": args.learning_rate}
1987     return {key: value for key, value in mapping.items() if value is not None}
1988
1989

```

```

1990 def _exploration_overrides(args: argparse.Namespace) -> dict:
1991     mapping = {
1992         "epsilon_decay_env_steps": args.epsilon_decay_env_steps,
1993         "epsilon_final": args.epsilon_final,
1994         "freeway_up_bias": args.freeway_up_bias,
1995     }
1996     return {key: value for key, value in mapping.items() if value is not None}
1997
1998
1999 def _selected_seeds(args: argparse.Namespace) -> list[int]:
2000     if args.seeds is not None:
2001         return args.seeds
2002     if args.variant == "all":
2003         return list(DEFAULT_SEEDS)
2004     return [args.seed]
2005
2006
2007 def _build_configs(args: argparse.Namespace, variant: str, seeds: list[int]) -> list[RunConfig]:
2008     return [
2009         build_run_config(
2010             env_id=args.env,
2011             variant_name=variant,
2012             seed=seed,
2013             output_root=Path(args.output_root),
2014             training_overrides=_training_overrides(args),
2015             dqn_overrides=_dqn_overrides(args),
2016             optimizer_overrides=_optimizer_overrides(args),
2017             exploration_overrides=_exploration_overrides(args),
2018         )
2019         for seed in seeds
2020     ]
2021
2022
2023 def _run_config(config: RunConfig) -> TrainResult:
2024     summary = DQNTrainer(config).run()
2025     return TrainResult(
2026         run_id=config.run_id,
2027         run_dir=config.run_dir,
2028         variant=config.variant.name,
2029         seed=config.seed,
2030         final_eval_return_mean=summary["performance"]["final_eval_return_mean"],
2031     )
2032
2033
2034 def _print_completed(result: TrainResult) -> None:
2035     print(
2036         f"Completed {result.run_id}: final_eval_return_mean={result.final_eval_return_mean}",
2037         flush=True,
2038     )
2039
2040
2041 def _print_failed(config: RunConfig, exc: BaseException) -> None:
2042     print(f"Failed {config.run_id} (variant={config.variant.name}, seed={config.seed}): {exc}", file=sys.stderr)
2043
2044
2045 def _resolve_max_workers(args: argparse.Namespace, num_configs: int) -> int:
2046     if num_configs <= 0:
2047         return 0
2048     if args.max_seed_workers is None:
2049         return num_configs
2050     return min(args.max_seed_workers, num_configs)
2051
2052
2053 def _run_configs_serial(configs: list[RunConfig]) -> int:
2054     failures = 0
2055     for config in configs:
2056         print(f"Starting {config.run_id} -> {config.run_dir}", flush=True)
2057         try:
2058             _print_completed(_run_config(config))
2059         except Exception as exc:
2060             failures += 1
2061             _print_failed(config, exc)
2062     return failures
2063

```



```

2064
2065 def _run_configs_parallel(configs: list[RunConfig], max_workers: int) -> int:
2066     failures = 0
2067     for config in configs:
2068         print(f"Starting {config.run_id} -> {config.run_dir}", flush=True)
2069         context = mp.get_context("spawn")
2070         with ProcessPoolExecutor(max_workers=max_workers, mp_context=context) as executor:
2071             future_to_config = {executor.submit(_run_config, config): config for config in configs}
2072             for future in as_completed(future_to_config):
2073                 config = future_to_config[future]
2074                 try:
2075                     _print_completed(future.result())
2076                 except Exception as exc:
2077                     failures += 1
2078                     _print_failed(config, exc)
2079     return failures
2080
2081
2082 def main(argv: list[str] | None = None) -> None:
2083     args = parse_args(argv)
2084     variants = VARIANT_NAMES if args.variant == "all" else (args.variant,)
2085     seeds = _selected_seeds(args)
2086     total_failures = 0
2087     for variant in variants:
2088         configs = _build_configs(args, variant, seeds)
2089         max_workers = _resolve_max_workers(args, len(configs))
2090         if args.serial or max_workers == 1:
2091             total_failures += _run_configs_serial(configs)
2092         else:
2093             total_failures += _run_configs_parallel(configs, max_workers)
2094     if total_failures:
2095         raise SystemExit(1)
2096
2097
2098 if __name__ == "__main__":
2099     main()
2100
2101
2102
2103 =====
2104 FILE: trainer.py
2105 =====
2106 """Training loop for LunarLander DQN ablations."""
2107
2108 from __future__ import annotations
2109
2110 from dataclasses import asdict, replace
2111 from datetime import datetime, timezone
2112 from pathlib import Path
2113 from typing import Any
2114
2115 import numpy as np
2116 import torch
2117
2118 from rl_analysis.agent import DQNAgent
2119 from rl_analysis.config import RunConfig
2120 from rl_analysis.envs import (
2121     environment_config_dict,
2122     freeway_episode_metrics,
2123     freeway_up_biased_action_probabilities,
2124     lunarlander_episode_metrics,
2125     make_env,
2126 )
2127 from rl_analysis.logging_utils import RunLoggers, collect_system_metrics, write_run_config
2128 from rl_analysis.networks import build_q_network, network_architecture_dict
2129 from rl_analysis.replay import ReplayBuffer, Transition, replay_diagnostics, transition_to_batch
2130 from rl_analysis.utils import (
2131     action_entropy,
2132     count_parameters,
2133     git_info,
2134     hardware_info,
2135     numeric_stats,
2136     read_jsonl,
2137     resolve_device,

```

```

238     rolling_mean,
239     rolling_std,
240     set_global_seeds,
241     software_info,
242     to_jsonable,
243     write_json,
244 )
245
246
247 class DQNTrainer:
248     def __init__(self, config: RunConfig, *, repo_root: Path | None = None):
249         self.config = config
250         self.repo_root = repo_root or Path.cwd()
251         self.rng = np.random.default_rng(config.seed)
252         self.eval_rng = np.random.default_rng(config.seed + 10_000)
253         set_global_seeds(config.seed)
254         self.device = resolve_device(config.training.device)
255
256         self.env = make_env(config.env, seed=config.seed)
257         self.eval_env = make_env(config.env, seed=config.seed + 1_000)
258         observation_shape = tuple(int(x) for x in self.env.observation_space.shape)
259         action_dim = int(self.env.action_space.n)
260         self.observation_storage_dtype = np.uint8 if config.env.observation_type == "image" else np.float32
261
262         online_network = build_q_network(observation_shape, action_dim, config.network)
263         target_network = (
264             build_q_network(observation_shape, action_dim, config.network) if config.variant.use_target_network else None
265         )
266         num_parameters = count_parameters(online_network)
267         self.config.network = replace(self.config.network, num_parameters=num_parameters)
268         self.agent = DQNAgent(
269             online_network=online_network,
270             target_network=target_network,
271             variant=config.variant,
272             dqn_config=config.dqn,
273             optimizer_config=config.optimizer,
274             device=self.device,
275         )
276         self.training_random_action_probabilities = (
277             freeway_up_biased_action_probabilities(action_dim, config.exploration.freeway_up_bias)
278             if config.env.env_id == "ALE/Freeway-v5" and config.exploration.freeway_up_bias > 0.0
279             else None
280         )
281         self.replay_buffer = (
282             ReplayBuffer(
283                 config.dqn.replay_buffer_size,
284                 observation_shape,
285                 observation_dtype=self.observation_storage_dtype,
286             )
287             if config.variant.use_replay_buffer
288             else None
289         )
290         self.run_dir = config.run_dir
291         self.checkpoint_dir = self.run_dir / "checkpoints"
292         self.episode_returns: list[float] = []
293         self.episode_lengths: list[int] = []
294         self.eval_records: list[dict[str, Any]] = []
295         self.update_records: list[dict[str, Any]] = []
296         self.max_score_so_far = 0.0
297         self.failure_flags = {
298             "nan_detected": False,
299             "inf_detected": False,
300             "q_value_explosion": False,
301             "loss_explosion": False,
302             "gradient_explosion": False,
303             "early_terminated": False,
304         }
305         self.wall_time_start = 0.0
306
307     def run(self) -> dict[str, Any]:
308         import time
309
310         self.run_dir.mkdir(parents=True, exist_ok=True)
311         self.checkpoint_dir.mkdir(parents=True, exist_ok=True)

```

```

2112 self.config.status = "running"
2113 self._write_run_config()
2114 self.wall_time_start = time.perf_counter()
2115 updates_done = 0
2116 last_update_time = self.wall_time_start
2117
2118 next_eval_step = self.config.training.eval_frequency_env_steps
2119 next_checkpoint_step = self.config.training.checkpoint_frequency_env_steps
2120 next_system_step = self.config.training.system_metrics_frequency_env_steps
2121 latest_eval: dict[str, Any] | None = None
2122
2123 obs, _ = self.env.reset(seed=self.config.seed)
2124 episode_return = 0.0
2125 episode_length = 0
2126 episode_index = 0
2127 episode_start = time.perf_counter()
2128 episode_action_counts = np.zeros(self.agent.action_dim, dtype=np.int64)
2129 global_step = 0
2130
2131 with RunLoggers(self.run_dir) as loggers:
2132     while global_step < self.config.training.total_env_steps:
2133         epsilon = self.config.exploration.epsilon_at(global_step)
2134         action = self.agent.select_action(
2135             obs,
2136             epsilon,
2137             self.rng,
2138             random_action_probabilities=self.training_random_action_probabilities,
2139         )
2140         next_obs, reward, terminated, truncated, _info = self.env.step(action)
2141         global_step += 1
2142         episode_return += float(reward)
2143         episode_length += 1
2144         episode_action_counts[action] += 1
2145
2146         transition = Transition(
2147             state=np.asarray(obs, dtype=self.observation_storage_dtype),
2148             action=action,
2149             reward=float(reward),
2150             next_state=np.asarray(next_obs, dtype=self.observation_storage_dtype),
2151             terminated=bool(terminated),
2152             truncated=bool(truncated),
2153             global_step=global_step,
2154             episode_index=episode_index,
2155         )
2156         batch_for_update = None
2157         if self.replay_buffer is not None:
2158             self.replay_buffer.add(transition)
2159             if self._should_update(global_step):
2160                 for _ in range(self.config.dqn.gradient_steps_per_train):
2161                     batch_for_update = self.replay_buffer.sample(self.config.dqn.batch_size, self.rng)
2162                     update_metrics = self._update_and_log(
2163                         batch_for_update, global_step, loggers, updates_done
2164                     )
2165                     updates_done += 1
2166                     self._merge_failure_flags(update_metrics)
2167             elif self._should_update(global_step):
2168                 batch_for_update = transition_to_batch(transition)
2169                 update_metrics = self._update_and_log(batch_for_update, global_step, loggers, updates_done)
2170                 updates_done += 1
2171                 self._merge_failure_flags(update_metrics)
2172
2173         self.agent.maybe_update_target(global_step)
2174
2175         if terminated or truncated:
2176             episode_wall_time = time.perf_counter() - episode_start
2177             self.episode_returns.append(float(episode_return))
2178             self.episode_lengths.append(int(episode_length))
2179             if self.config.env.env_id == "ALE/Freeway-v5":
2180                 self.max_score_so_far = max(self.max_score_so_far, float(episode_return))
2181             game_specific = self._episode_game_metrics(
2182                 episode_return=episode_return,
2183                 episode_length=episode_length,
2184                 action_counts=episode_action_counts,
2185                 final_observation=np.asarray(next_obs, dtype=self.observation_storage_dtype),

```

```

2286         terminated=bool(terminated),
2287         truncated=bool(truncated),
2288     )
2289     loggers.train_episode.write(
2290         {
2291             "run_id": self.config.run_id,
2292             "env_id": self.config.env.env_id,
2293             "variant": self.config.variant.name,
2294             "seed": self.config.seed,
2295             "episode_index": episode_index,
2296             "global_env_step": global_step,
2297             "episode_return": float(episode_return),
2298             "episode_length": episode_length,
2299             "episode_wall_time_sec": episode_wall_time,
2300             "epsilon": epsilon,
2301             "learning_rate": self.config.optimizer.learning_rate,
2302             "replay_buffer_size": len(self.replay_buffer) if self.replay_buffer is not None else 0,
2303             "rolling_return_10": rolling_mean(self.episode_returns, 10),
2304             "rolling_return_50": rolling_mean(self.episode_returns, 50),
2305             "rolling_return_100": rolling_mean(self.episode_returns, 100),
2306             "rolling_length_100": rolling_mean([float(x) for x in self.episode_lengths], 100),
2307             "terminated": bool(terminated),
2308             "truncated": bool(truncated),
2309             "termination_reason": "truncated" if truncated else "environment",
2310             "game_specific": game_specific,
2311         }
2312     )
2313     episode_index += 1
2314     if (
2315         self.config.training.max_episodes is not None
2316         and episode_index >= self.config.training.max_episodes
2317     ):
2318         break
2319     obs, _ = self.env.reset()
2320     episode_return = 0.0
2321     episode_length = 0
2322     episode_start = time.perf_counter()
2323     episode_action_counts = np.zeros(self.agent.action_dim, dtype=np.int64)
2324 else:
2325     obs = next_obs
2326
2327     if global_step >= next_eval_step or global_step >= self.config.training.total_env_steps:
2328         latest_eval = self.evaluate(global_step, len(self.eval_records))
2329         self.eval_records.append(latest_eval)
2330         loggers.eval.write(latest_eval)
2331         next_eval_step += self.config.training.eval_frequency_env_steps
2332
2333     if global_step >= next_checkpoint_step or global_step >= self.config.training.total_env_steps:
2334         checkpoint_metrics = self.save_checkpoint(global_step, latest_eval)
2335         loggers.checkpoint.write(checkpoint_metrics)
2336         next_checkpoint_step += self.config.training.checkpoint_frequency_env_steps
2337
2338     if global_step >= next_system_step or global_step >= self.config.training.total_env_steps:
2339         elapsed = time.perf_counter() - self.wall_time_start
2340         since_update = max(1e-9, time.perf_counter() - last_update_time)
2341         loggers.system.write(
2342             collect_system_metrics(
2343                 run_id=self.config.run_id,
2344                 global_env_step=global_step,
2345                 device=self.device,
2346                 wall_time_start=self.wall_time_start,
2347                 env_steps_per_second=global_step / elapsed if elapsed > 0 else None,
2348                 updates_per_second=updates_done / since_update if updates_done else 0.0,
2349                 replay_buffer_memory_gb=self.replay_buffer.memory_gb()
2350                 if self.replay_buffer is not None
2351                 else None,
2352             )
2353         )
2354         next_system_step += self.config.training.system_metrics_frequency_env_steps
2355
2356     summary = self.build_summary(global_step=global_step, total_updates=updates_done)
2357     write_json(self.run_dir / "summary.json", summary)
2358     self.config.status = "completed"
2359     self.config.datetime_end = datetime.now(timezone.utc).isoformat()

```

```

2360     self._write_run_config()
2361     self.env.close()
2362     self.eval_env.close()
2363     return summary
2364
2365 def _episode_game_metrics(
2366     self,
2367     *,
2368     episode_return: float,
2369     episode_length: int,
2370     action_counts: np.ndarray,
2371     final_observation: np.ndarray,
2372     terminated: bool,
2373     truncated: bool,
2374 ) -> dict[str, Any]:
2375     if self.config.env.env_id == "ALE/Freeway-v5":
2376         return freeway_episode_metrics(
2377             episode_return=episode_return,
2378             episode_length=episode_length,
2379             action_counts=action_counts,
2380             max_score_so_far=self.max_score_so_far,
2381         )
2382     return lunarlander_episode_metrics(
2383         episode_return=episode_return,
2384         episode_length=episode_length,
2385         action_counts=action_counts,
2386         final_observation=np.asarray(final_observation, dtype=np.float32),
2387         terminated=terminated,
2388         truncated=truncated,
2389     )
2390
2391 def _should_update(self, global_step: int) -> bool:
2392     if global_step < self.config.dqn.learning_starts:
2393         return False
2394     if global_step % self.config.dqn.train_frequency_env_steps != 0:
2395         return False
2396     if self.replay_buffer is not None and len(self.replay_buffer) < self.config.dqn.batch_size:
2397         return False
2398     return True
2399
2400 def _update_and_log(
2401     self, batch, global_step: int, loggers: RunLoggers, updates_done: int
2402 ) -> dict[str, Any]:
2403     update_metrics = self.agent.update(batch, global_step)
2404     update_metrics.update(
2405         {
2406             "run_id": self.config.run_id,
2407             "env_id": self.config.env.env_id,
2408             "variant": self.config.variant.name,
2409             "seed": self.config.seed,
2410             "replay": replay_diagnostics(
2411                 batch=batch,
2412                 replay_buffer=self.replay_buffer,
2413                 current_step=global_step,
2414                 replay_enabled=self.config.variant.use_replay_buffer,
2415             ),
2416         }
2417     )
2418     if update_metrics["update_index"] % self.config.training.update_log_frequency == 0:
2419         self.update_records.append(update_metrics)
2420         loggers.train_update.write(update_metrics)
2421     return update_metrics
2422
2423 def _merge_failure_flags(self, update_metrics: dict[str, Any]) -> None:
2424     failures = update_metrics.get("failure_diagnostics", {})
2425     for key in self.failure_flags:
2426         self.failure_flags[key] = bool(self.failure_flags[key] or failures.get(key, False))
2427
2428 def evaluate(self, global_step: int, eval_index: int) -> dict[str, Any]:
2429     returns = []
2430     lengths = []
2431     entropies = []
2432     action_counts_rows = []
2433     game_metric_rows = []

```

```

2434
2435 for episode in range(self.config.training.num_eval_episodes):
2436     obs, _ = self.eval_env.reset(seed=self.config.seed + 100_000 + eval_index * 1_000 + episode)
2437     done = False
2438     episode_return = 0.0
2439     episode_length = 0
2440     action_counts = np.zeros(self.agent.action_dim, dtype=np.int64)
2441     final_obs = obs
2442     terminated = False
2443     truncated = False
2444     while not done:
2445         action = self.agent.select_action(obs, self.config.exploration.eval_epsilon, self.eval_rng)
2446         obs, reward, terminated, truncated, _info = self.eval_env.step(action)
2447         final_obs = obs
2448         done = bool(terminated or truncated)
2449         episode_return += float(reward)
2450         episode_length += 1
2451         action_counts[action] += 1
2452
2453     metrics = self._eval_episode_game_metrics(
2454         episode_return=episode_return,
2455         episode_length=episode_length,
2456         action_counts=action_counts,
2457         final_observation=np.asarray(final_obs, dtype=self.observation_storage_dtype),
2458         terminated=bool(terminated),
2459         truncated=bool(truncated),
2460     )
2461     returns.append(float(episode_return))
2462     lengths.append(float(episode_length))
2463     entropies.append(action_entropy(action_counts))
2464     action_counts_rows.append(action_counts)
2465     game_metric_rows.append(metrics)
2466
2467     return_stats = numeric_stats(returns)
2468     length_stats = numeric_stats(lengths)
2469     best_so_far = max([*self.eval_records, {"return_mean": -float("inf")}], key=lambda row: row["return_mean"])
2470     if return_stats["mean"] is not None and return_stats["mean"] >= best_so_far["return_mean"]:
2471         best_eval_return = return_stats["mean"]
2472         best_eval_step = global_step
2473     else:
2474         best_eval_return = best_so_far["return_mean"]
2475         best_eval_step = best_so_far["global_env_step"]
2476
2477     action_counts_mean = np.mean(np.stack(action_counts_rows, axis=0), axis=0)
2478     payload = {
2479         "run_id": self.config.run_id,
2480         "env_id": self.config.env.env_id,
2481         "variant": self.config.variant.name,
2482         "seed": self.config.seed,
2483         "global_env_step": global_step,
2484         "eval_index": eval_index,
2485         "num_eval_episodes": self.config.training.num_eval_episodes,
2486         "eval_epsilon": self.config.exploration.eval_epsilon,
2487         "return_mean": return_stats["mean"],
2488         "return_std": return_stats["std"],
2489         "return_median": return_stats["median"],
2490         "return_min": return_stats["min"],
2491         "return_max": return_stats["max"],
2492         "return_p25": return_stats["p25"],
2493         "return_p75": return_stats["p75"],
2494         "episode_length_mean": length_stats["mean"],
2495         "episode_length_std": length_stats["std"],
2496         "best_eval_return_mean_so_far": best_eval_return,
2497         "best_eval_step_so_far": best_eval_step,
2498         "action_entropy_mean": float(np.mean(entropies)),
2499         "action_entropy_std": float(np.std(entropies)),
2500         "action_counts_mean": action_counts_mean.tolist(),
2501     }
2502     if self.config.env.env_id == "ALE/Freeway-v5":
2503         payload["freeway_eval"] = self._freeway_eval_summary(returns, game_metric_rows)
2504     else:
2505         payload["lunarlander_eval"] = self._lunarlander_eval_summary(returns, game_metric_rows)
2506     return payload
2507

```

```

2308 def _eval_episode_game_metrics(
2309     self,
2310     *,
2311     episode_return: float,
2312     episode_length: int,
2313     action_counts: np.ndarray,
2314     final_observation: np.ndarray,
2315     terminated: bool,
2316     truncated: bool,
2317 ) -> dict[str, Any]:
2318     if self.config.env.env_id == "ALE/Freeway-v5":
2319         return freeway_episode_metrics(
2320             episode_return=episode_return,
2321             episode_length=episode_length,
2322             action_counts=action_counts,
2323             max_score_so_far=max(self.max_score_so_far, float(episode_return)),
2324         )
2325     return lunarlander_episode_metrics(
2326         episode_return=episode_return,
2327         episode_length=episode_length,
2328         action_counts=action_counts,
2329         final_observation=np.asarray(final_observation, dtype=np.float32),
2330         terminated=terminated,
2331         truncated=truncated,
2332     )
2333
2334 def _lunarlander_eval_summary(
2335     self, returns: list[float], metric_rows: list[dict[str, Any]]
2336 ) -> dict[str, Any]:
2337     both_legs = [float(row["both_legs_contact"]) for row in metric_rows]
2338     one_leg = [float(row["one_leg_contact"]) for row in metric_rows]
2339     return {
2340         "success_rate": float(np.mean([1.0 if r >= 200.0 else 0.0 for r in returns])),
2341         "success_definition": "return >= 200",
2342         "landing_success_rate": float(np.mean([row["landing_success"] for row in metric_rows])),
2343         "crash_rate": float(np.mean([row["crash"] for row in metric_rows])),
2344         "timeout_rate": float(np.mean([row["timeout"] for row in metric_rows])),
2345         "main_engine_action_fraction_mean": float(
2346             np.mean([row["main_engine_action_fraction"] for row in metric_rows])
2347         ),
2348         "side_engine_action_fraction_mean": float(
2349             np.mean([row["side_engine_action_fraction"] for row in metric_rows])
2350         ),
2351         "both_legs_contact_rate": float(np.mean(both_legs)),
2352         "one_leg_contact_rate": float(np.mean(one_leg)),
2353         "no_leg_contact_rate": float(1.0 - np.mean(both_legs) - np.mean(one_leg)),
2354     }
2355
2356 def _freeway_eval_summary(self, returns: list[float], metric_rows: list[dict[str, Any]]) -> dict[str, Any]:
2357     score_stats = numeric_stats(returns)
2358     threshold_rates = {
2359         str(threshold): float(np.mean([1.0 if score >= threshold else 0.0 for score in returns]))
2360         for threshold in self.config.env.score_thresholds
2361     }
2362     return {
2363         "score_mean": score_stats["mean"],
2364         "score_std": score_stats["std"],
2365         "score_median": score_stats["median"],
2366         "score_min": score_stats["min"],
2367         "score_max": score_stats["max"],
2368         "zero_score_rate": float(np.mean([row["zero_score_episode"] for row in metric_rows])),
2369         "action_up_fraction_mean": float(np.mean([row["action_up_fraction"] for row in metric_rows])),
2370         "action_down_fraction_mean": float(np.mean([row["action_down_fraction"] for row in metric_rows])),
2371         "action_noop_fraction_mean": float(np.mean([row["action_noop_fraction"] for row in metric_rows])),
2372         "score_threshold_success_rates": threshold_rates,
2373     }
2374
2375 def save_checkpoint(self, global_step: int, latest_eval: dict[str, Any] | None) -> dict[str, Any]:
2376     checkpoint_path = self.checkpoint_dir / f"model_step_{global_step}.pt"
2377     payload = {
2378         "run_id": self.config.run_id,
2379         "global_env_step": global_step,
2380         "variant": self.config.variant.name,
2381         "seed": self.config.seed,

```

```

2382         "online_network_state_dict": self.agent.online_network.state_dict(),
2383         "target_network_state_dict": self.agent.target_network.state_dict()
2384         if self.agent.target_network is not None
2385         else None,
2386         "optimizer_state_dict": self.agent.optimizer.state_dict(),
2387         "config": to_jsonable(self.config),
2388     }
2389     torch.save(payload, checkpoint_path)
2390     model_size_mb = checkpoint_path.stat().st_size / 1024**2
2391     best_eval = None
2392     if latest_eval is not None:
2393         best_eval = latest_eval["return_mean"] == latest_eval["best_eval_return_mean_so_far"]
2394     return {
2395         "run_id": self.config.run_id,
2396         "global_env_step": global_step,
2397         "checkpoint_index": len(list(self.checkpoint_dir.glob("model_step_*.pt"))),
2398         "checkpoint_path": str(checkpoint_path.relative_to(self.run_dir)),
2399         "eval_return_mean_at_checkpoint": latest_eval.get("return_mean") if latest_eval else None,
2400         "eval_return_std_at_checkpoint": latest_eval.get("return_std") if latest_eval else None,
2401         "is_best_checkpoint_so_far": best_eval,
2402         "model_size_mb": float(model_size_mb),
2403         "optimizer_state_size_mb": None,
2404     }
2405
2406 def build_summary(self, *, global_step: int, total_updates: int) -> dict[str, Any]:
2407     eval_returns = [float(row["return_mean"]) for row in self.eval_records if row.get("return_mean") is not None]
2408     eval_steps = [int(row["global_env_step"]) for row in self.eval_records]
2409     final_eval = self.eval_records[-1] if self.eval_records else None
2410     best_eval_return = max(eval_returns) if eval_returns else None
2411     best_eval_step = eval_steps[int(np.argmax(eval_returns))] if eval_returns else None
2412     auc = float(np.trapezoid(eval_returns, eval_steps)) if len(eval_returns) >= 2 else 0.0
2413     normalized_auc = auc / max(1, self.config.training.total_env_steps)
2414     first_threshold = self._first_threshold_step(eval_returns, eval_steps)
2415     sustained_threshold = self._first_sustained_threshold_step(eval_returns, eval_steps)
2416     collapse_count, largest_drop, largest_drop_fraction = self._collapse_metrics(eval_returns)
2417     wall_time_total = 0.0
2418     if self.wall_time_start:
2419         import time
2420
2421         wall_time_total = time.perf_counter() - self.wall_time_start
2422
2423     logged_update_rows = read_jsonl(self.run_dir / "train_update_metrics.jsonl")
2424     update_source = logged_update_rows or self.update_records
2425     last_updates = update_source[-max(1, len(update_source) // 10) :] if update_source else []
2426     q_max_values = [
2427         row.get("q_values", {}).get("online_q_max")
2428         for row in update_source
2429         if row.get("q_values", {}).get("online_q_max") is not None
2430     ]
2431
2432     return {
2433         "run_id": self.config.run_id,
2434         "env_id": self.config.env.env_id,
2435         "variant": self.config.variant.name,
2436         "seed": self.config.seed,
2437         "training_completed": True,
2438         "total_env_steps": global_step,
2439         "total_ale_frames": self._estimated_ale_frames(global_step),
2440         "total_episodes": len(self.episode_returns),
2441         "total_updates": total_updates,
2442         "performance": {
2443             "final_train_return_mean_last_100": rolling_mean(self.episode_returns, 100),
2444             "final_train_return_std_last_100": rolling_std(self.episode_returns, 100),
2445             "final_eval_return_mean": final_eval.get("return_mean") if final_eval else None,
2446             "final_eval_return_std": final_eval.get("return_std") if final_eval else None,
2447             "final_eval_return_median": final_eval.get("return_median") if final_eval else None,
2448             "best_eval_return_mean": best_eval_return,
2449             "best_eval_step": best_eval_step,
2450             "area_under_eval_curve": auc,
2451             "normalized_area_under_eval_curve": normalized_auc,
2452         },
2453         "sample_efficiency": {
2454             "threshold": self.config.env.success_threshold,
2455             "first_step_reaching_threshold": first_threshold,

```



```

2656         "sustained_threshold_window": self.config.training.sustained_threshold_window,
2657         "first_step_sustained_threshold": sustained_threshold,
2658         "never_reached_threshold": first_threshold is None,
2659     },
2660     "stability": {
2661         "rolling_return_std_mean": rolling_std(self.episode_returns, min(100, len(self.episode_returns)))
2662         if self.episode_returns
2663         else None,
2664         "rolling_return_std_max": self._rolling_return_std_max(),
2665         "eval_return_std_across_time": float(np.std(eval_returns)) if eval_returns else None,
2666         "catastrophic_collapse_count": collapse_count,
2667         "largest_eval_drop": largest_drop,
2668         "largest_eval_drop_fraction": largest_drop_fraction,
2669         "collapse_definition": "eval return mean drops by >= 30% from previous best after crossing threshold",
2670     },
2671     "optimization": self._optimization_summary(last_updates),
2672     "q_diagnostics": self._q_summary(last_updates, q_max_values),
2673     "replay_diagnostics": self._replay_summary(last_updates),
2674     "target_network_diagnostics": self._target_summary(last_updates),
2675     "compute": {
2676         "wall_time_total_sec": wall_time_total,
2677         "env_steps_per_second": global_step / wall_time_total if wall_time_total > 0 else None,
2678         "updates_per_second": total_updates / wall_time_total if wall_time_total > 0 else None,
2679     },
2680     **(
2681         {"freeway": self._freeway_summary(eval_returns, eval_steps, final_eval)}
2682         if self.config.env.env_id == "ALE/Freeway-v5"
2683         else {}
2684     ),
2685     "failure_diagnostics": dict(self.failure_flags),
2686 }
2687
2688 def _estimated_ale_frames(self, env_steps: int) -> int | None:
2689     if self.config.env.frame_skip is None:
2690         return None
2691     return int(env_steps * self.config.env.frame_skip)
2692
2693 def _freeway_summary(
2694     self,
2695     eval_returns: list[float],
2696     eval_steps: list[int],
2697     final_eval: dict[str, Any] | None,
2698 ) -> dict[str, Any]:
2699     threshold_steps = self._first_steps_for_thresholds(eval_returns, eval_steps, self.config.env.score_thresholds)
2700     final_freeway = final_eval.get("freeway_eval", {}) if final_eval else {}
2701     return {
2702         "final_eval_score_mean": final_eval.get("return_mean") if final_eval else None,
2703         "best_eval_score_mean": max(eval_returns) if eval_returns else None,
2704         "final_zero_score_rate": final_freeway.get("zero_score_rate"),
2705         "score_thresholds": list(self.config.env.score_thresholds),
2706         "first_step_reaching_score_thresholds": threshold_steps,
2707         "reference_random_score": self.config.env.reference_random_score,
2708         "reference_human_score": self.config.env.reference_human_score,
2709         "reference_dqn_score": self.config.env.reference_dqn_score,
2710     }
2711
2712 def _first_steps_for_thresholds(
2713     self, eval_returns: list[float], eval_steps: list[int], thresholds: tuple[float, ...]
2714 ) -> dict[str, int | None]:
2715     first_steps: dict[str, int | None] = {}
2716     for threshold in thresholds:
2717         first_steps[str(threshold)] = None
2718         for value, step in zip(eval_returns, eval_steps, strict=False):
2719             if value >= threshold:
2720                 first_steps[str(threshold)] = step
2721                 break
2722     return first_steps
2723
2724 def _first_threshold_step(self, eval_returns: list[float], eval_steps: list[int]) -> int | None:
2725     threshold = self.config.env.success_threshold
2726     if threshold is None:
2727         return None
2728     for value, step in zip(eval_returns, eval_steps, strict=False):
2729         if value >= threshold:

```

```

2730         return step
2731     return None
2732
2733 def _first_sustained_threshold_step(self, eval_returns: list[float], eval_steps: list[int]) -> int | None:
2734     threshold = self.config.env.success_threshold
2735     if threshold is None:
2736         return None
2737     window = self.config.training.sustained_threshold_window
2738     for start in range(0, max(0, len(eval_returns) - window + 1)):
2739         chunk = eval_returns[start : start + window]
2740         if all(value >= threshold for value in chunk):
2741             return eval_steps[start]
2742     return None
2743
2744 def _collapse_metrics(self, eval_returns: list[float]) -> tuple[int, float | None, float | None]:
2745     threshold = self.config.env.success_threshold
2746     if threshold is None or not eval_returns:
2747         return 0, None, None
2748     best = -float("inf")
2749     crossed = False
2750     count = 0
2751     largest_drop = 0.0
2752     largest_fraction = 0.0
2753     for value in eval_returns:
2754         if value >= threshold:
2755             crossed = True
2756         if crossed and best > 0:
2757             drop = max(0.0, best - value)
2758             fraction = drop / best if best else 0.0
2759             if fraction >= 0.30:
2760                 count += 1
2761             largest_drop = max(largest_drop, drop)
2762             largest_fraction = max(largest_fraction, fraction)
2763     best = max(best, value)
2764     return count, float(largest_drop), float(largest_fraction)
2765
2766 def _rolling_return_std_max(self) -> float | None:
2767     if not self.episode_returns:
2768         return None
2769     window = min(100, len(self.episode_returns))
2770     values = [
2771         float(np.std(self.episode_returns[max(0, index - window + 1) : index + 1]))
2772         for index in range(len(self.episode_returns))
2773     ]
2774     return float(np.max(values))
2775
2776 def _optimization_summary(self, rows: list[dict[str, Any]]) -> dict[str, Any]:
2777     if not rows:
2778         return {
2779             "td_loss_mean_last_10pct": None,
2780             "td_error_abs_mean_last_10pct": None,
2781             "td_error_abs_p95_last_10pct": None,
2782             "grad_norm_mean_last_10pct": None,
2783             "grad_norm_max": None,
2784             "gradient_clip_fraction": None,
2785         }
2786     return {
2787         "td_loss_mean_last_10pct": float(np.mean([row["loss"]["td_loss_mean"] for row in rows])),
2788         "td_error_abs_mean_last_10pct": float(np.mean([row["loss"]["td_error_abs_mean"] for row in rows])),
2789         "td_error_abs_p95_last_10pct": float(np.mean([row["loss"]["td_error_abs_p95"] for row in rows])),
2790         "grad_norm_mean_last_10pct": float(np.mean([row["optimization"]["grad_norm"] for row in rows])),
2791         "grad_norm_max": float(np.max([row["optimization"]["grad_norm"] for row in rows])),
2792         "gradient_clip_fraction": float(np.mean([row["optimization"]["grad_norm_clipped"] for row in rows])),
2793     }
2794
2795 def _q_summary(self, rows: list[dict[str, Any]], q_max_values: list[float]) -> dict[str, Any]:
2796     if not rows:
2797         return {
2798             "online_q_mean_last_10pct": None,
2799             "online_q_max_mean_last_10pct": None,
2800             "target_q_mean_last_10pct": None,
2801             "mean_q_overestimation_proxy_last_10pct": None,
2802             "max_q_value_seen": max(q_max_values) if q_max_values else None,
2803         }

```

```

2804     return {
2805         "online_q_mean_last_10pct": float(np.mean([row["q_values"]["online_q_mean"] for row in rows])),
2806         "online_q_max_mean_last_10pct": float(np.mean([row["q_values"]["online_q_max_mean"] for row in rows])),
2807         "target_q_mean_last_10pct": float(np.mean([row["q_values"]["target_q_mean"] for row in rows])),
2808         "mean_q_overestimation_proxy_last_10pct": float(
2809             np.mean([row["q_values"]["q_overestimation_proxy"] for row in rows])
2810         ),
2811         "max_q_value_seen": max(q_max_values) if q_max_values else None,
2812     }
2813
2814 def _replay_summary(self, rows: list[dict[str, Any]]) -> dict[str, Any]:
2815     if not rows:
2816         return {
2817             "replay_enabled": self.config.variant.use_replay_buffer,
2818             "final_buffer_size": len(self.replay_buffer) if self.replay_buffer is not None else 0,
2819             "sample_age_mean_last_10pct": None,
2820             "sample_consecutive_transition_fraction_last_10pct": None,
2821         }
2822     return {
2823         "replay_enabled": self.config.variant.use_replay_buffer,
2824         "final_buffer_size": len(self.replay_buffer) if self.replay_buffer is not None else 0,
2825         "sample_age_mean_last_10pct": float(
2826             np.mean([row["replay"]["sample_age_mean"] for row in rows if row["replay"]["sample_age_mean"] is not None
2827 ↪ ]))
2828         ),
2829         "sample_consecutive_transition_fraction_last_10pct": float(
2830             np.mean(
2831                 [
2832                     row["replay"]["sample_consecutive_transition_fraction"]
2833                     for row in rows
2834                     if row["replay"]["sample_consecutive_transition_fraction"] is not None
2835                 ]
2836             )
2837         )
2838     )
2839     if any(row["replay"]["sample_consecutive_transition_fraction"] is not None for row in rows)
2840     else None,
2841     }
2842
2843 def _target_summary(self, rows: list[dict[str, Any]]) -> dict[str, Any]:
2844     if not rows:
2845         return {
2846             "target_network_enabled": self.config.variant.use_target_network,
2847             "target_update_count": self.agent.target_update_count if self.agent.target_network is not None else None,
2848             "online_target_param_l2_mean": None,
2849             "online_target_param_l2_max": None,
2850         }
2851     values = [
2852         row["target_network"]["online_target_param_l2"]
2853         for row in rows
2854         if row["target_network"]["online_target_param_l2"] is not None
2855     ]
2856     return {
2857         "target_network_enabled": self.config.variant.use_target_network,
2858         "target_update_count": self.agent.target_update_count if self.agent.target_network is not None else None,
2859         "online_target_param_l2_mean": float(np.mean(values)) if values else None,
2860         "online_target_param_l2_max": float(np.max(values)) if values else None,
2861     }
2862
2863 def _write_run_config(self) -> None:
2864     algorithm = {
2865         "base": "DQN",
2866         "use_target_network": self.config.variant.use_target_network,
2867         "use_replay_buffer": self.config.variant.use_replay_buffer,
2868         "use_double_dqn": self.config.variant.use_double_dqn,
2869         "use Dueling_network": self.config.variant.use_dueling_network,
2870         "use_prioritized_replay": False,
2871         "use_noisy_network": False,
2872         "use_reward_clipping": self.config.variant.use_reward_clipping or self.config.env.reward_clipping,
2873     }
2874     payload = {
2875         "run_id": self.config.run_id,
2876         "experiment_group": self.config.experiment_group,

```

```

2877         "env_id": self.config.env.env_id,
2878         "variant": self.config.variant.name,
2879         "variant_label": self.config.variant.label,
2880         "seed": self.config.seed,
2881         "datetime_start": self.config.datetime_start,
2882         "datetime_end": self.config.datetime_end,
2883         "status": self.config.status,
2884         "algorithm": algorithm,
2885         "training_budget": {
2886             "total_env_steps": self.config.training.total_env_steps,
2887             "max_episodes": self.config.training.max_episodes,
2888             "max_episode_steps": self.config.env.max_episode_steps,
2889             "num_seeds": self.config.training.num_seeds,
2890             "eval_frequency_env_steps": self.config.training.eval_frequency_env_steps,
2891             "checkpoint_frequency_env_steps": self.config.training.checkpoint_frequency_env_steps,
2892             "agent_step_to_ale_frame_multiplier": self.config.env.frame_skip,
2893             "estimated_total_ale_frames": self._estimated_ale_frames(self.config.training.total_env_steps),
2894         },
2895         "environment": environment_config_dict(self.config.env),
2896         "network": network_architecture_dict(self.config.network, self.config.network.num_parameters or 0),
2897         "optimizer": asdict(self.config.optimizer),
2898         "dqn_hyperparameters": asdict(self.config.dqn),
2899         "exploration": asdict(self.config.exploration),
2900         "software": software_info(),
2901         "hardware": hardware_info(self.device),
2902         "code": git_info(self.repo_root),
2903     }
2904     write_run_config(self.run_dir, payload)
2905
2906
2907
2908     =====
2909     FILE: utils.py
2910     =====
2911     """Small shared utilities."""
2912
2913     from __future__ import annotations
2914
2915     import json
2916     import os
2917     import platform
2918     import random
2919     import subprocess
2920     import sys
2921     from dataclasses import asdict, is_dataclass
2922     from pathlib import Path
2923     from typing import Any, Iterable
2924
2925     import numpy as np
2926     import torch
2927
2928
2929     def _mps_backend_available() -> bool:
2930         return bool(hasattr(torch.backends, "mps") and torch.backends.mps.is_available())
2931
2932
2933     def _mps_backend_built() -> bool:
2934         return bool(hasattr(torch.backends, "mps") and torch.backends.mps.is_built())
2935
2936
2937     def _mps_unavailable_message() -> str:
2938         if not hasattr(torch.backends, "mps"):
2939             return "MPS device requested, but this PyTorch install does not expose torch.backends.mps."
2940         if not torch.backends.mps.is_built():
2941             return "MPS device requested, but this PyTorch install was not built with MPS support."
2942         return (
2943             "MPS device requested, but MPS is not available. Use an Apple Silicon Mac with Metal support, "
2944             "macOS 14.0 or newer, and an MPS-capable PyTorch install."
2945         )
2946
2947
2948     def set_global_seeds(seed: int) -> None:
2949         os.environ["PYTHONHASHSEED"] = str(seed)
2950         random.seed(seed)

```

```

2951 np.random.seed(seed)
2952 torch.manual_seed(seed)
2953 if torch.cuda.is_available():
2954     torch.cuda.manual_seed(seed)
2955     torch.cuda.manual_seed_all(seed)
2956 if hasattr(torch, "mps") and torch.mps.is_available():
2957     torch.mps.manual_seed(seed)
2958 if hasattr(torch.backends, "cudnn"):
2959     torch.backends.cudnn.benchmark = False
2960
2961
2962 def resolve_device(device_name: str) -> torch.device:
2963     if device_name == "auto":
2964         if torch.cuda.is_available():
2965             return torch.device("cuda")
2966         if _mps_backend_available():
2967             return torch.device("mps")
2968         return torch.device("cpu")
2969
2970     device = torch.device(device_name)
2971     if device.type == "mps" and not _mps_backend_available():
2972         raise RuntimeError(_mps_unavailable_message())
2973     return device
2974
2975
2976 def to_jsonable(value: Any) -> Any:
2977     if is_dataclass(value):
2978         return to_jsonable(asdict(value))
2979     if isinstance(value, Path):
2980         return str(value)
2981     if isinstance(value, dict):
2982         return {str(k): to_jsonable(v) for k, v in value.items()}
2983     if isinstance(value, (list, tuple)):
2984         return [to_jsonable(item) for item in value]
2985     if isinstance(value, np.ndarray):
2986         return value.tolist()
2987     if isinstance(value, np.generic):
2988         return value.item()
2989     if isinstance(value, torch.Tensor):
2990         if value.numel() == 1:
2991             return value.detach().cpu().item()
2992         return value.detach().cpu().tolist()
2993     return value
2994
2995
2996 def write_json(path: Path, payload: Any) -> None:
2997     path.parent.mkdir(parents=True, exist_ok=True)
2998     path.write_text(json.dumps(to_jsonable(payload), indent=2, sort_keys=True) + "\n", encoding="utf-8")
2999
3000
3001 def read_jsonl(path: Path) -> list[dict[str, Any]]:
3002     if not path.exists():
3003         return []
3004     rows = []
3005     with path.open("r", encoding="utf-8") as handle:
3006         for line in handle:
3007             line = line.strip()
3008             if line:
3009                 rows.append(json.loads(line))
3010     return rows
3011
3012
3013 def numeric_stats(values: Iterable[float]) -> dict[str, float | None]:
3014     arr = np.asarray(list(values), dtype=np.float64)
3015     if arr.size == 0:
3016         return {"mean": None, "std": None, "median": None, "min": None, "max": None, "p25": None, "p75": None}
3017     return {
3018         "mean": float(np.mean(arr)),
3019         "std": float(np.std(arr)),
3020         "median": float(np.median(arr)),
3021         "min": float(np.min(arr)),
3022         "max": float(np.max(arr)),
3023         "p25": float(np.percentile(arr, 25)),
3024         "p75": float(np.percentile(arr, 75)),

```

```

3025     }
3026
3027
3028 def rolling_mean(values: list[float], window: int) -> float | None:
3029     if not values:
3030         return None
3031     sample = values[-window:]
3032     return float(np.mean(sample))
3033
3034
3035 def rolling_std(values: list[float], window: int) -> float | None:
3036     if not values:
3037         return None
3038     sample = values[-window:]
3039     return float(np.std(sample))
3040
3041
3042 def action_entropy(action_counts: np.ndarray) -> float:
3043     total = int(np.sum(action_counts))
3044     if total == 0:
3045         return 0.0
3046     probs = action_counts.astype(np.float64) / total
3047     probs = probs[probs > 0]
3048     return float(-np.sum(probs * np.log(probs)))
3049
3050
3051 def count_parameters(model: torch.nn.Module) -> int:
3052     return int(sum(param.numel() for param in model.parameters()))
3053
3054
3055 def git_info(repo_root: Path) -> dict[str, Any]:
3056     def run_git(args: list[str]) -> str | None:
3057         try:
3058             result = subprocess.run(
3059                 ["git", *args],
3060                 cwd=repo_root,
3061                 check=True,
3062                 capture_output=True,
3063                 text=True,
3064             )
3065         except Exception:
3066             return None
3067         return result.stdout.strip()
3068
3069     branch = run_git(["rev-parse", "--abbrev-ref", "HEAD"])
3070     commit = run_git(["rev-parse", "HEAD"])
3071     dirty = run_git(["status", "--porcelain"])
3072     return {
3073         "git_commit": commit,
3074         "git_branch": branch,
3075         "dirty_working_tree": bool(dirty),
3076     }
3077
3078
3079 def software_info() -> dict[str, Any]:
3080     info = {
3081         "python_version": sys.version.split()[0],
3082         "platform": platform.platform(),
3083         "torch_version": torch.__version__,
3084         "numpy_version": np.__version__,
3085     }
3086     try:
3087         import gymnasium
3088
3089         info["gymnasium_version"] = gymnasium.__version__
3090     except Exception:
3091         info["gymnasium_version"] = None
3092     try:
3093         import ale_py
3094
3095         info["ale_py_version"] = ale_py.__version__
3096     except Exception:
3097         info["ale_py_version"] = None
3098     return info

```

```
3099
3100
3101 def hardware_info(device: torch.device) -> dict[str, Any]:
3102     gpu_name = None
3103     if device.type == "cuda" and torch.cuda.is_available():
3104         gpu_name = torch.cuda.get_device_name(device)
3105     elif device.type == "mps" and _mps_backend_available():
3106         gpu_name = "Apple Metal GPU"
3107     cpu_name = platform.processor() or os.uname().machine
3108     return {
3109         "device": str(device),
3110         "gpu_name": gpu_name,
3111         "cpu_name": cpu_name,
3112         "ram_gb": None,
3113         "mps_available": _mps_backend_available(),
3114         "mps_built": _mps_backend_built(),
3115     }
```

A.2 References

- [1] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, “Openai gym,” 2016. [Online]. Available: <https://arxiv.org/abs/1606.01540>
- [2] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, “The arcade learning environment: An evaluation platform for general agents,” *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013. [Online]. Available: <https://webdocs.cs.ualberta.ca/~mbowling/publications/b2hd-13jair-ale.html>
- [3] Farama Foundation, “Lunar lander,” https://gymnasium.farama.org/environments/box2d/lunar_lander/, 2026, accessed 2026-05-22.
- [4] ———, “Freeway,” <https://gymnasium.farama.org/v0.29.0/environments/atari/freeway/>, 2024, accessed 2026-05-22.
- [5] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. [Online]. Available: <https://doi.org/10.1038/nature14236>
- [6] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016, pp. 2094–2100. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/10295>
- [7] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Dueling network architectures for deep reinforcement learning,” in *Proceedings of the 33rd International Conference on Machine Learning*, vol. 48, 2016, pp. 1995–2003. [Online]. Available: <https://proceedings.mlr.press/v48/wangf16.html>